Codeforces 427C Checkposts

Grupo E - Problema B

Alunos:
Miguel Colares - 2422971
Renato Romano - 2426043
Vitor Dantas - 2422969
Orientador: Prof. Ricardo Carubbi
Disciplina: Resolução prob com grafos
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

Apresentação

Codeforces 427C Checkposts

Agenda

- Análise geral;
- Algoritmo de Kosaraju;
- Complexidades.

Análise geral

Checkposts

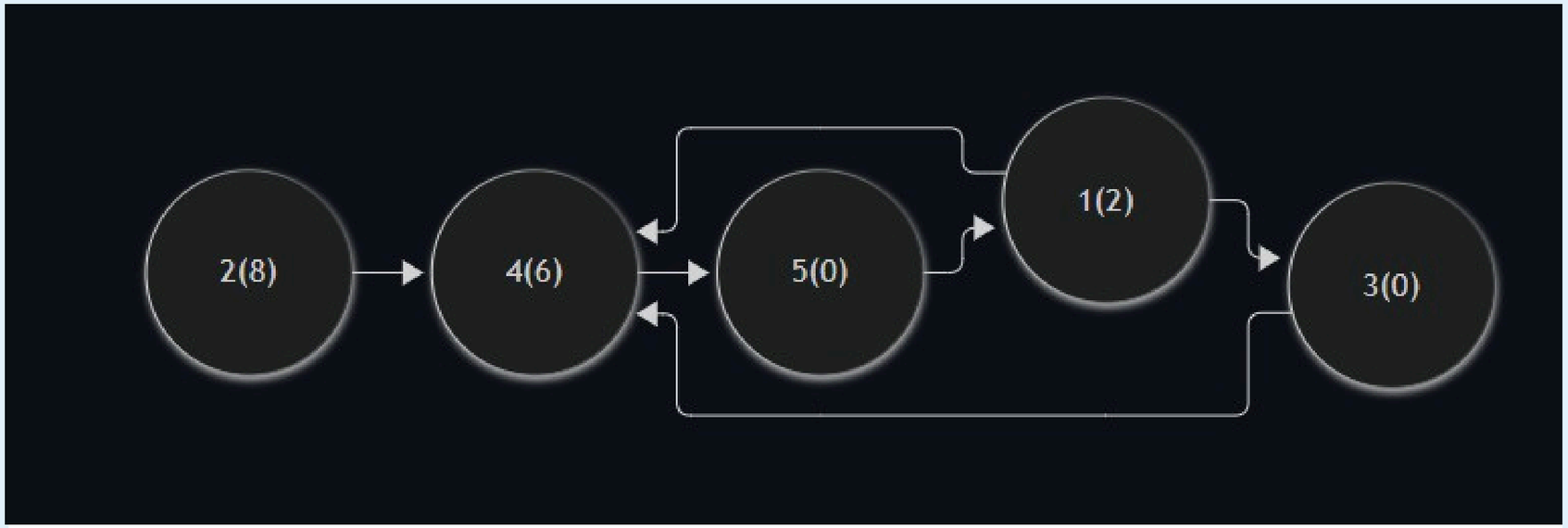
- Objetivo: encontrar o menor custo possível e, entre as soluções de menor custo, determinar a quantidade de formas que utilizam o menor número possível de postos.

input

```
5
2 8 0 6 0
6
1 4
1 3
2 4
3 4
4 5
5 1
```

output

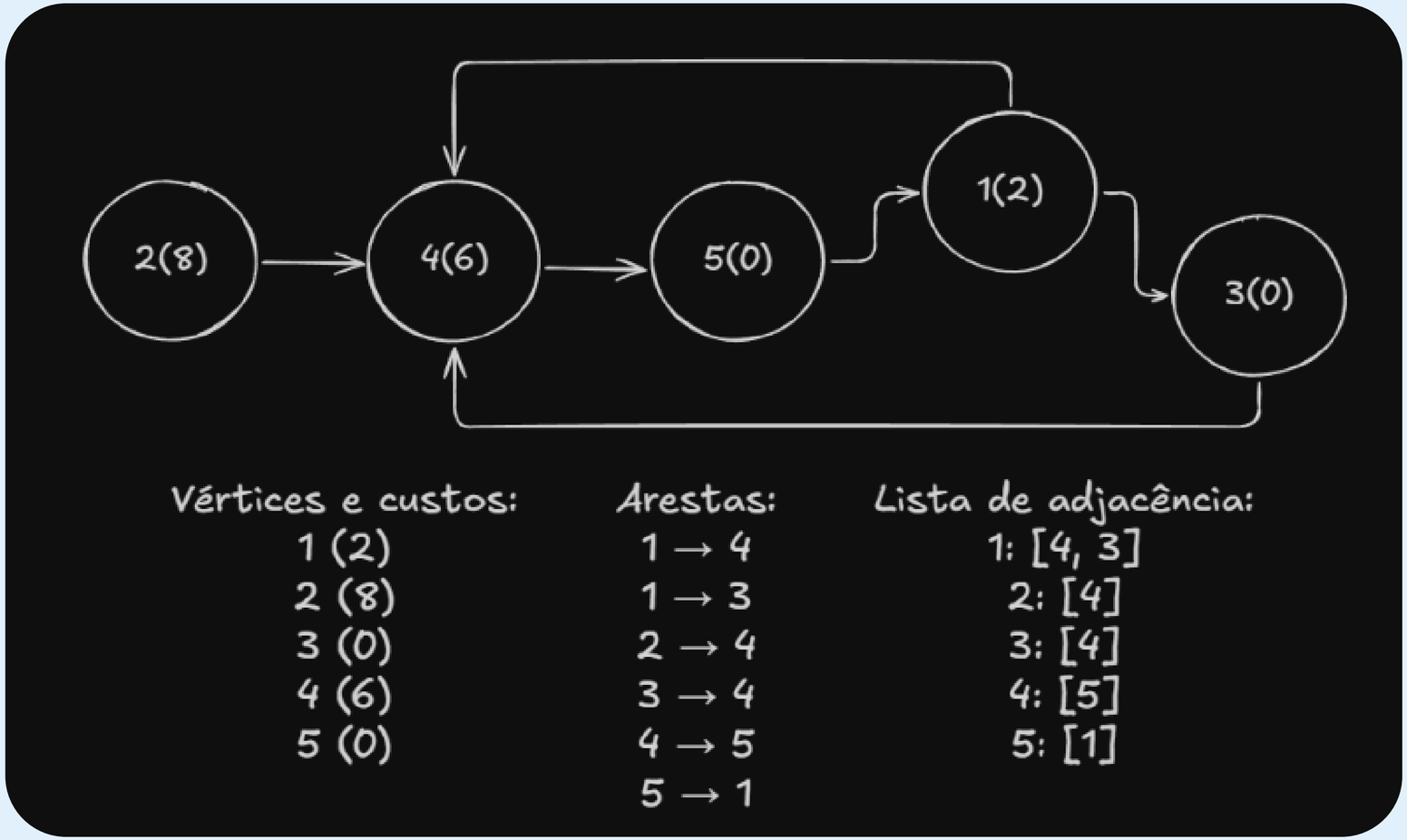
```
8 2
```



Algoritmo de Kosaraju

Checkposts

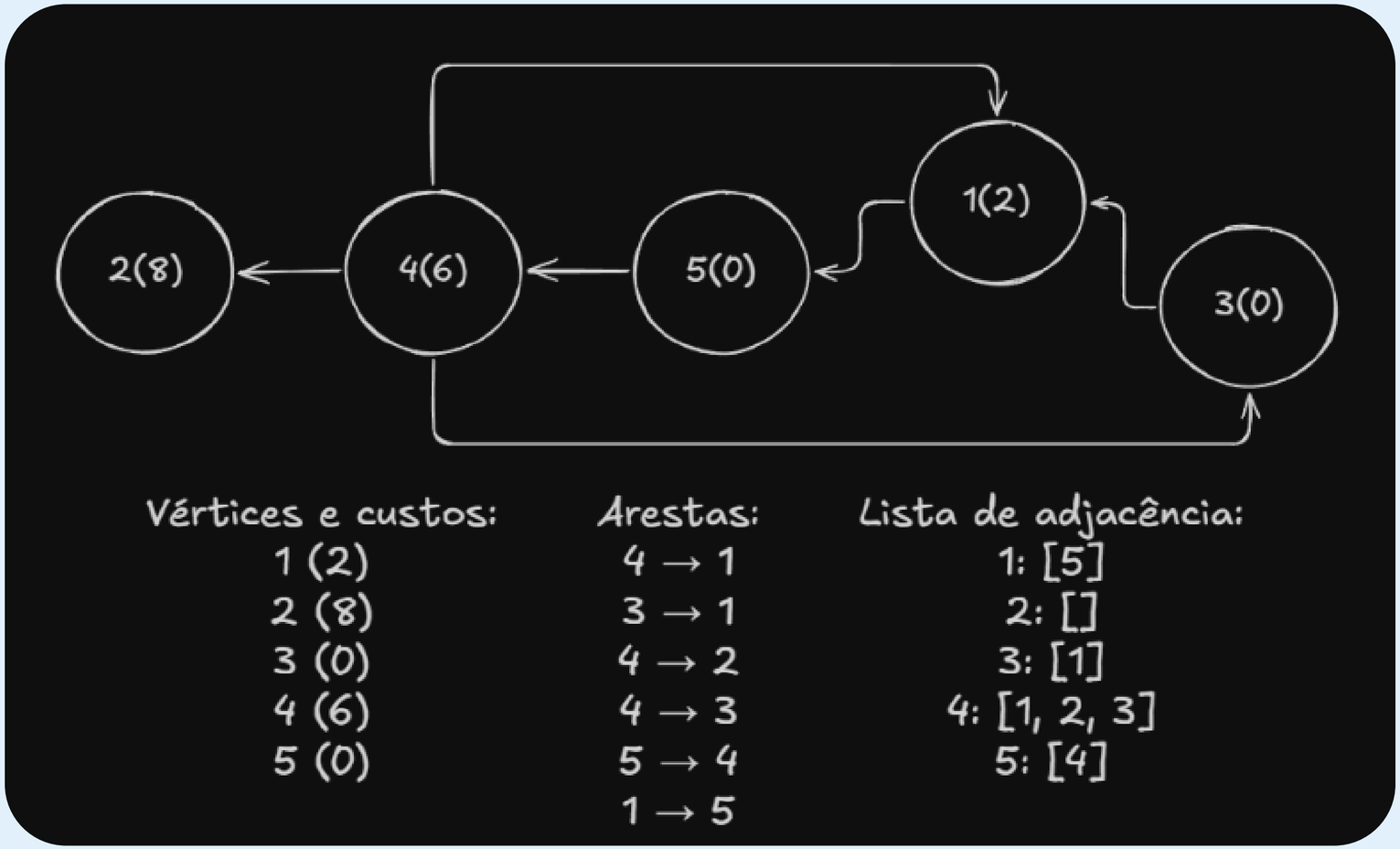
Passo 1: Grafo Original



Algoritmo de Kosaraju

Checkposts

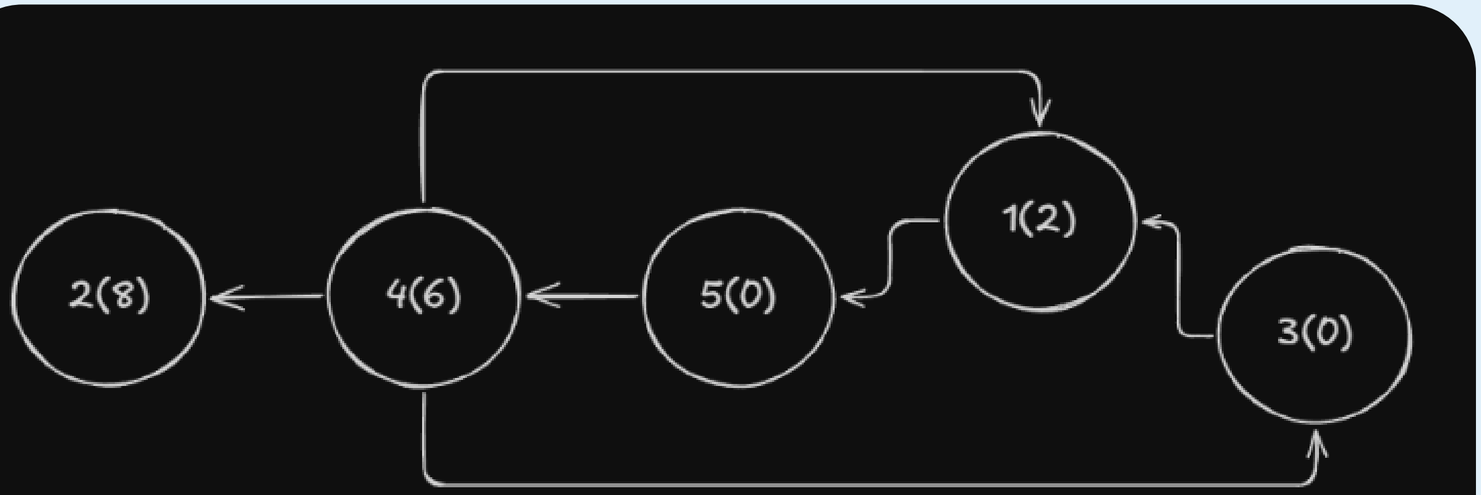
Passo 2: Inversão do grafo



Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Vértices e custos:

1 (2)
2 (8)
3 (0)
4 (6)
5 (0)

Arestas:

4 → 1
3 → 1
4 → 2
4 → 3
5 → 4
1 → 5

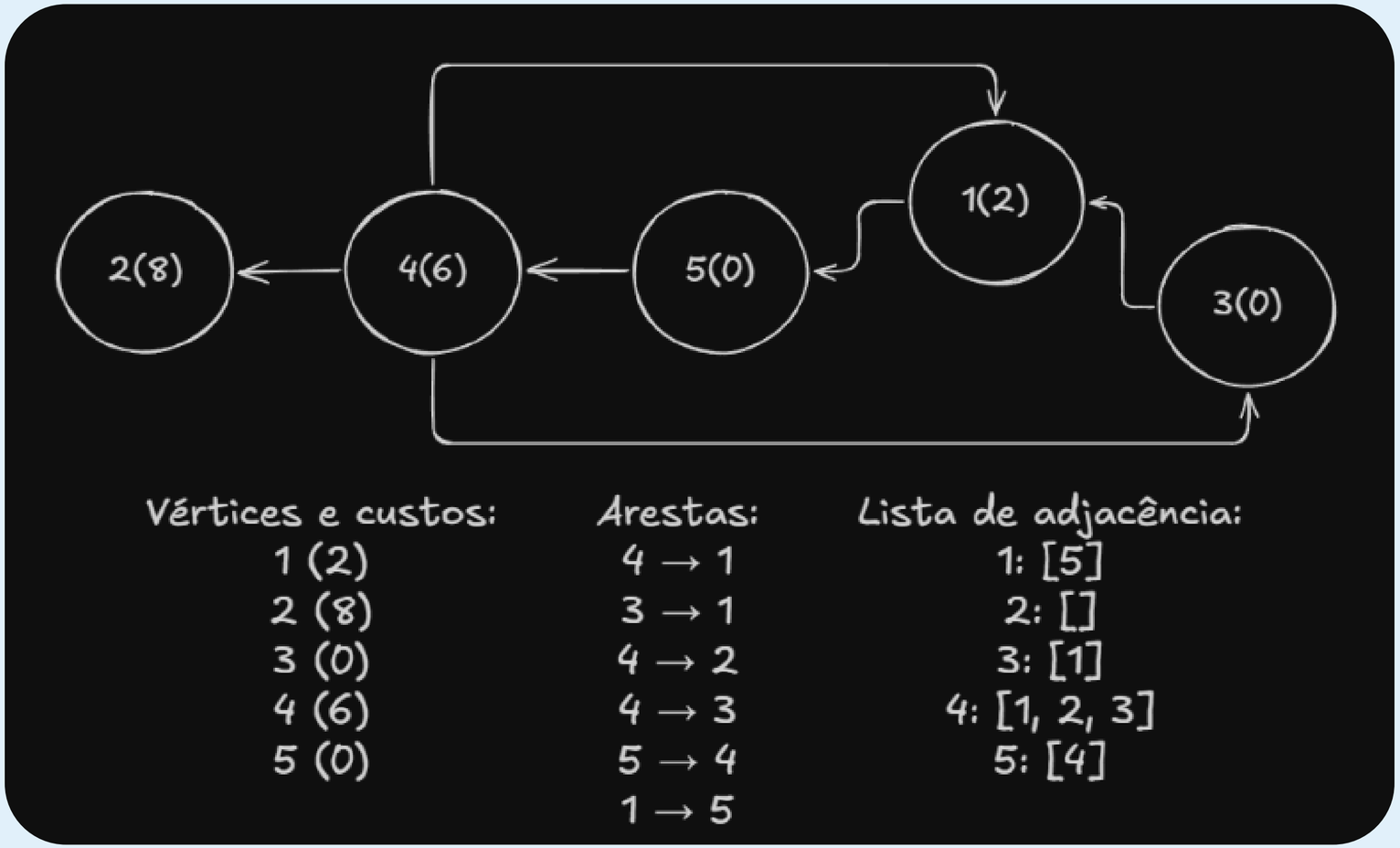
Lista de adjacência:

1: [5]
2: []
3: [1]
4: [1, 2, 3]
5: [4]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[ ]

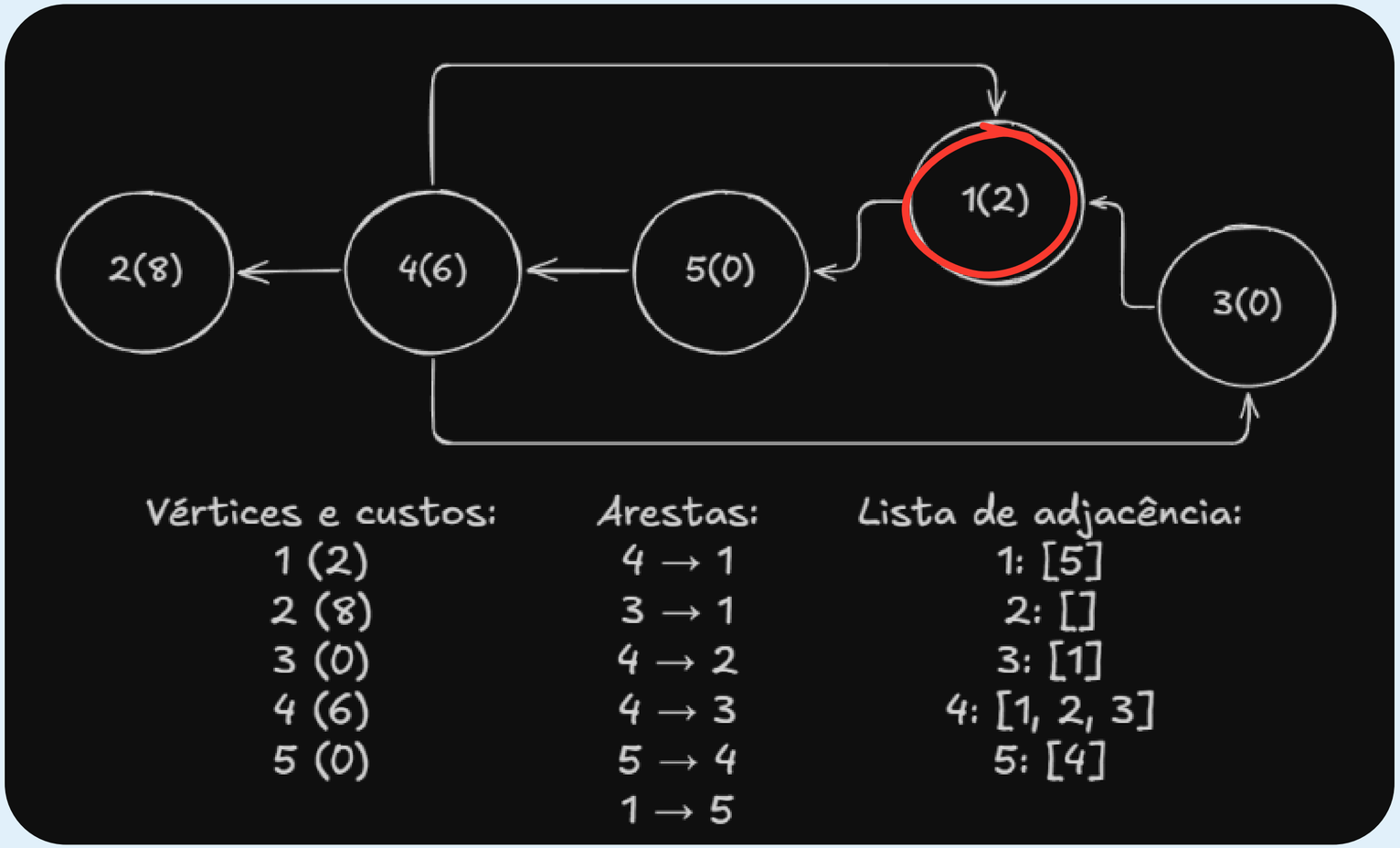
EdgeTo[ ]:
1: -
2:
3:
4:
5:

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1]

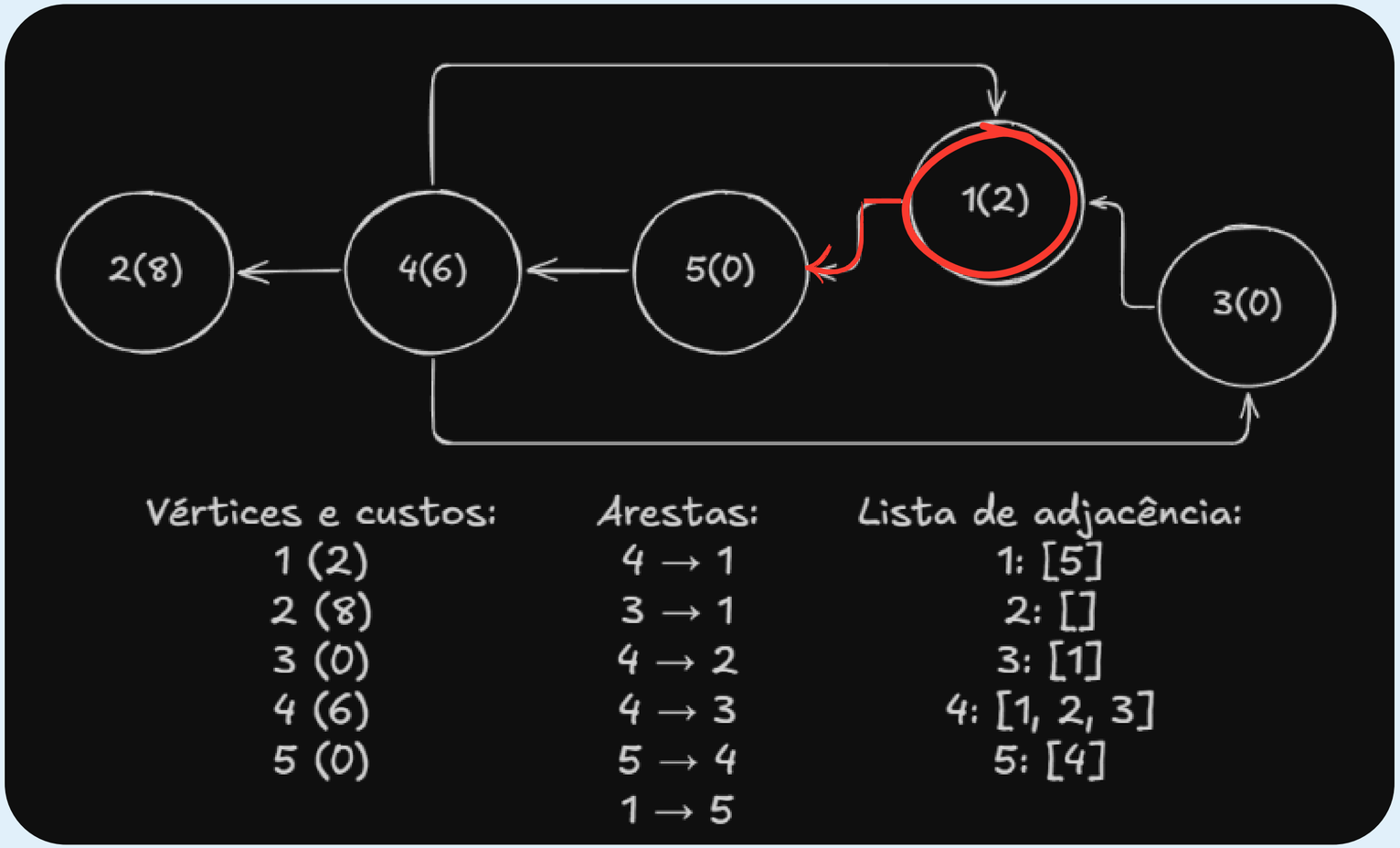
EdgeTo[ ]:
1: -
2:
3:
4:
5:

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1]

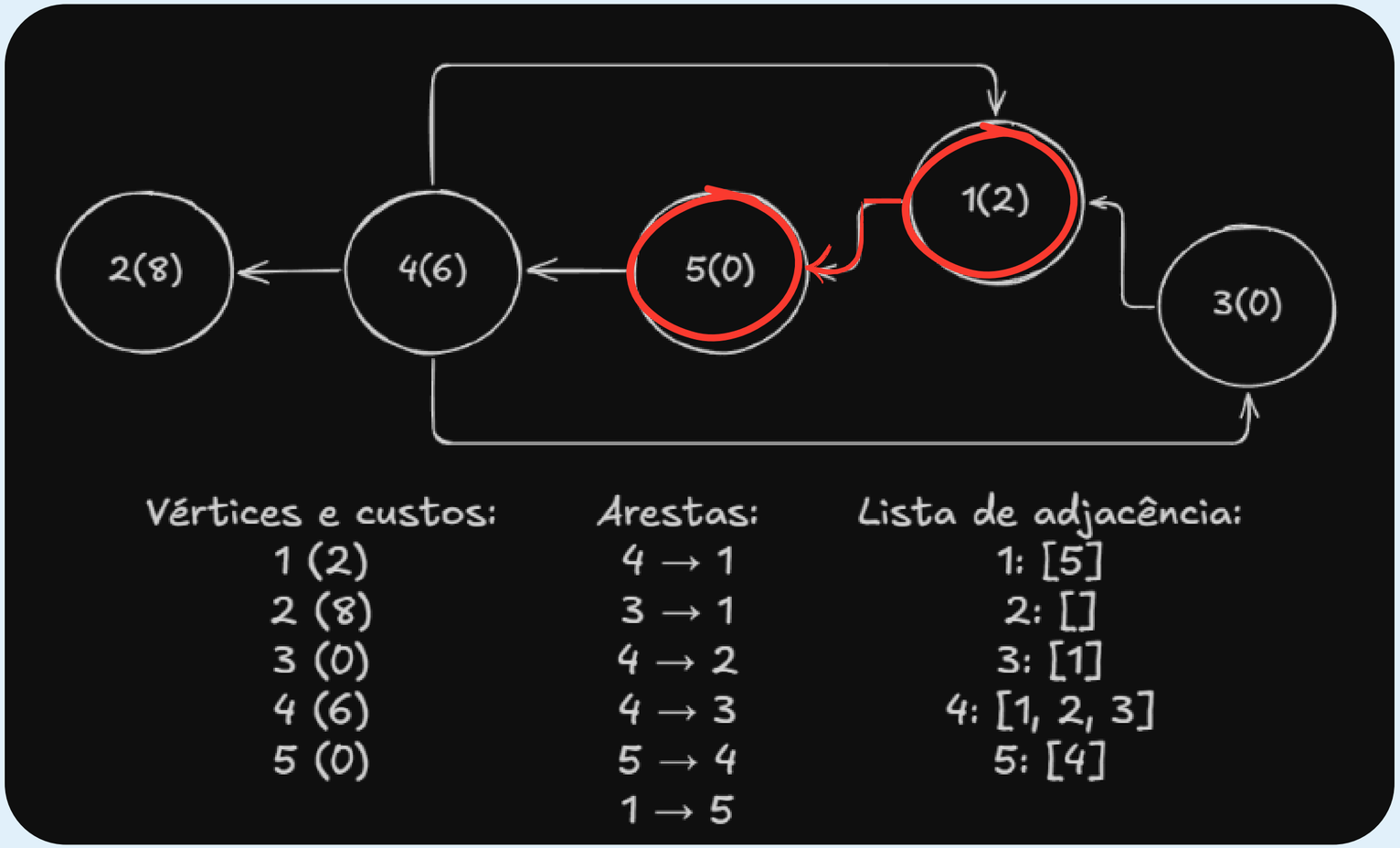
EdgeTo[ ]:
1: -
2:
3:
4:
5: 1

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 5]

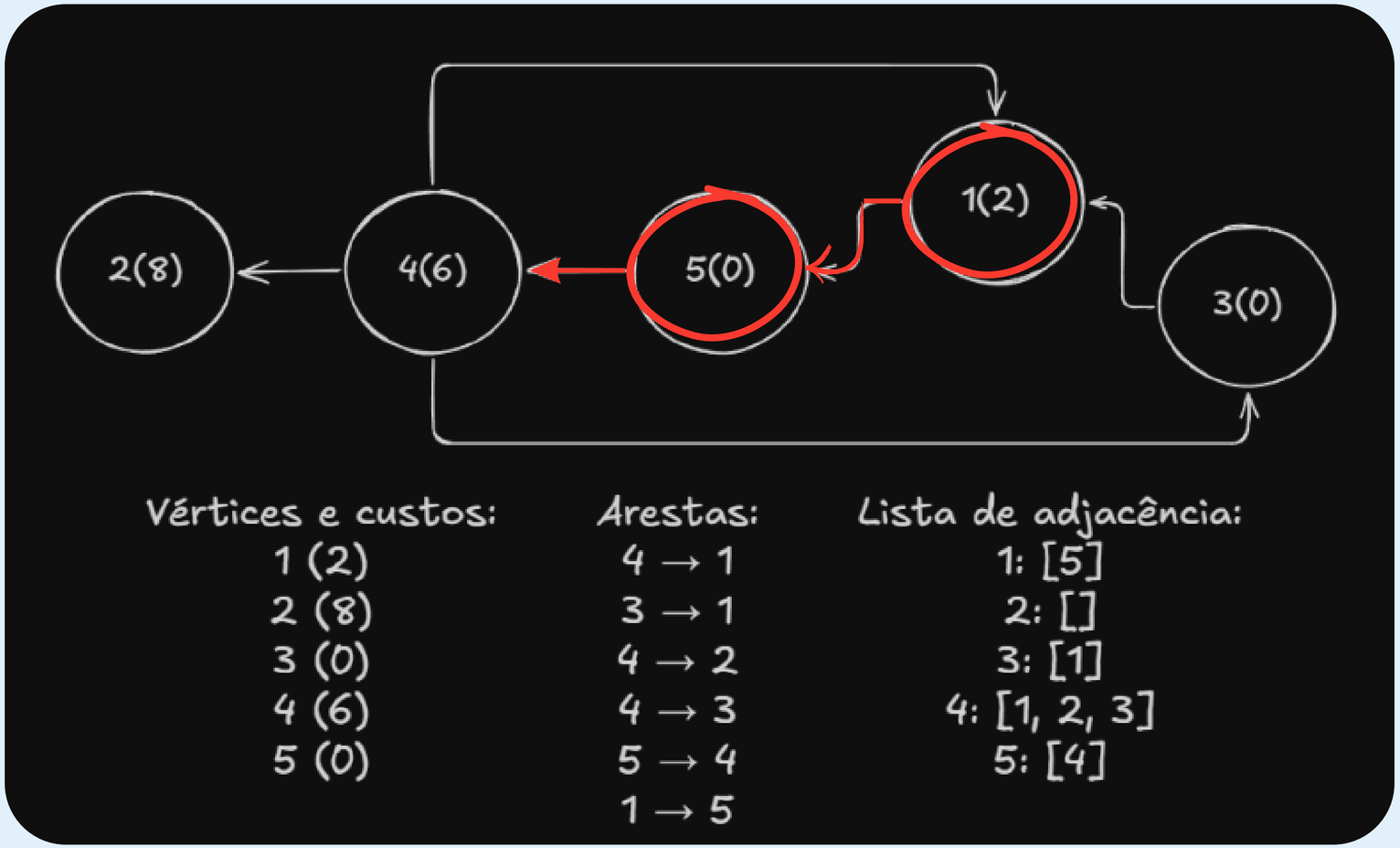
EdgeTo[ ]:
1: -
2:
3:
4:
5: 1

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 5]

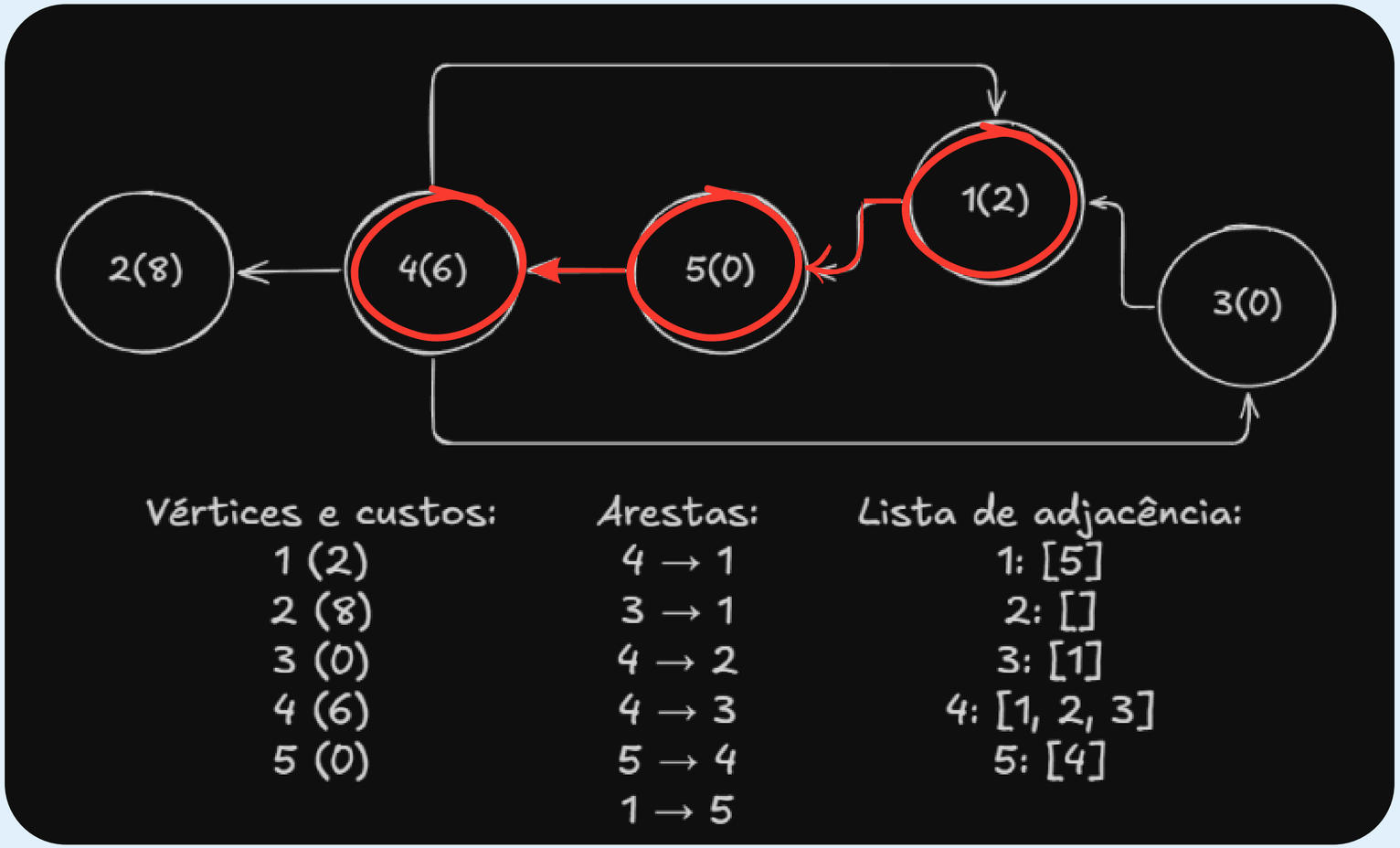
EdgeTo[ ]:
1: -
2:
3:
4: 5
5: 1

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 4, 5]

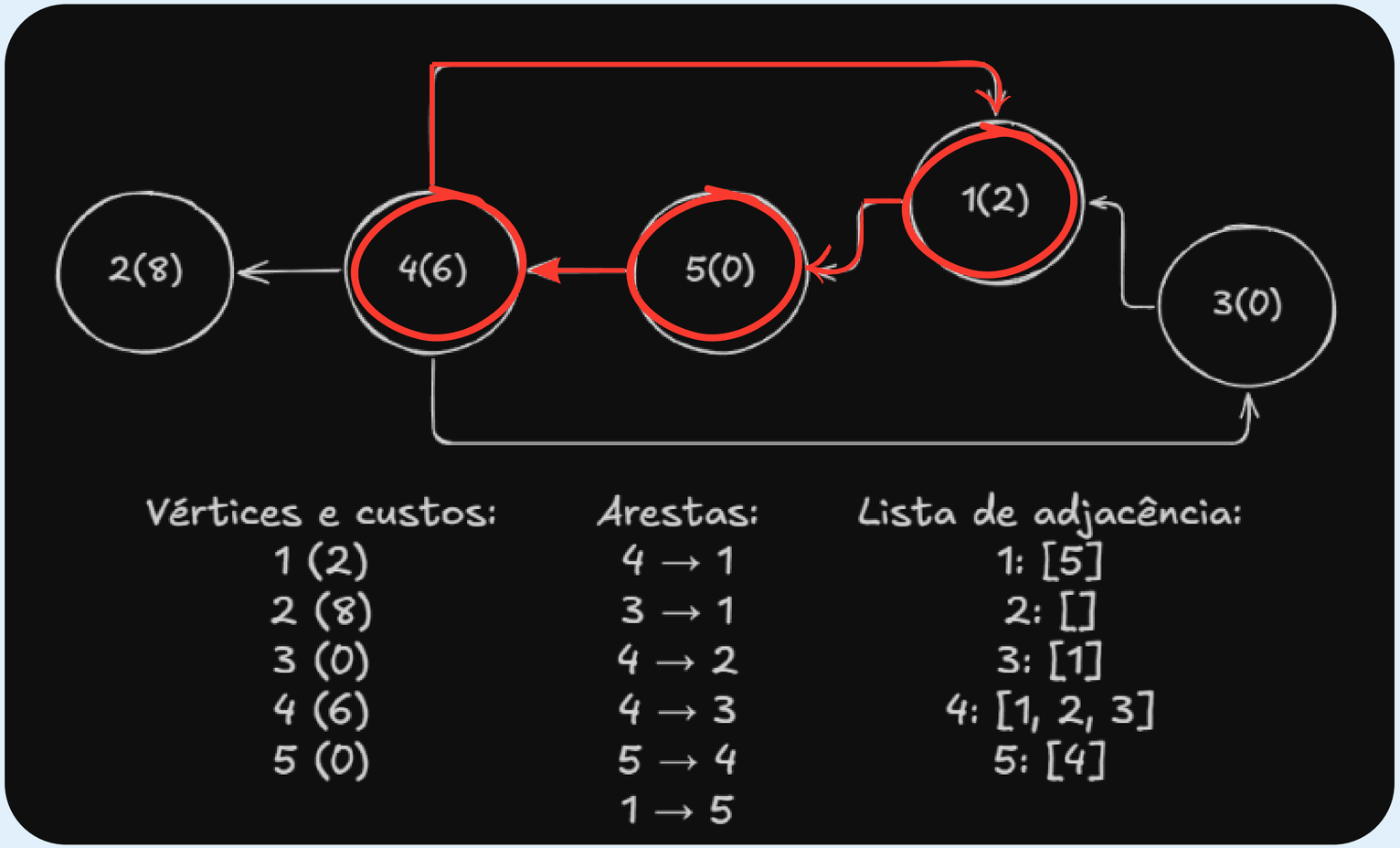
EdgeTo[ ]:
1: -
2:
3:
4: 5
5: 1

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 4, 5]

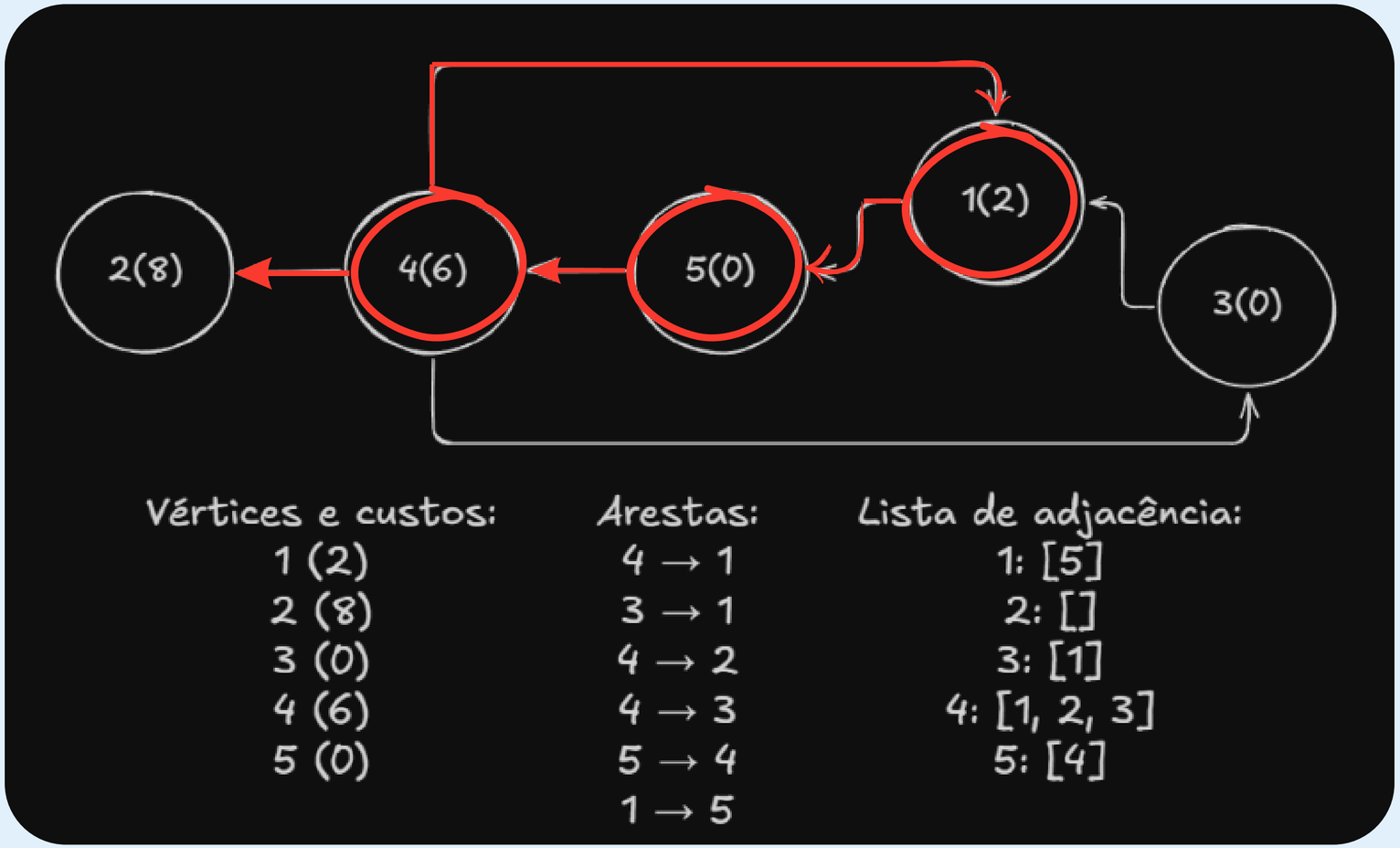
EdgeTo[ ]:
1: -
2:
3:
4: 5
5: 1

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 4, 5]

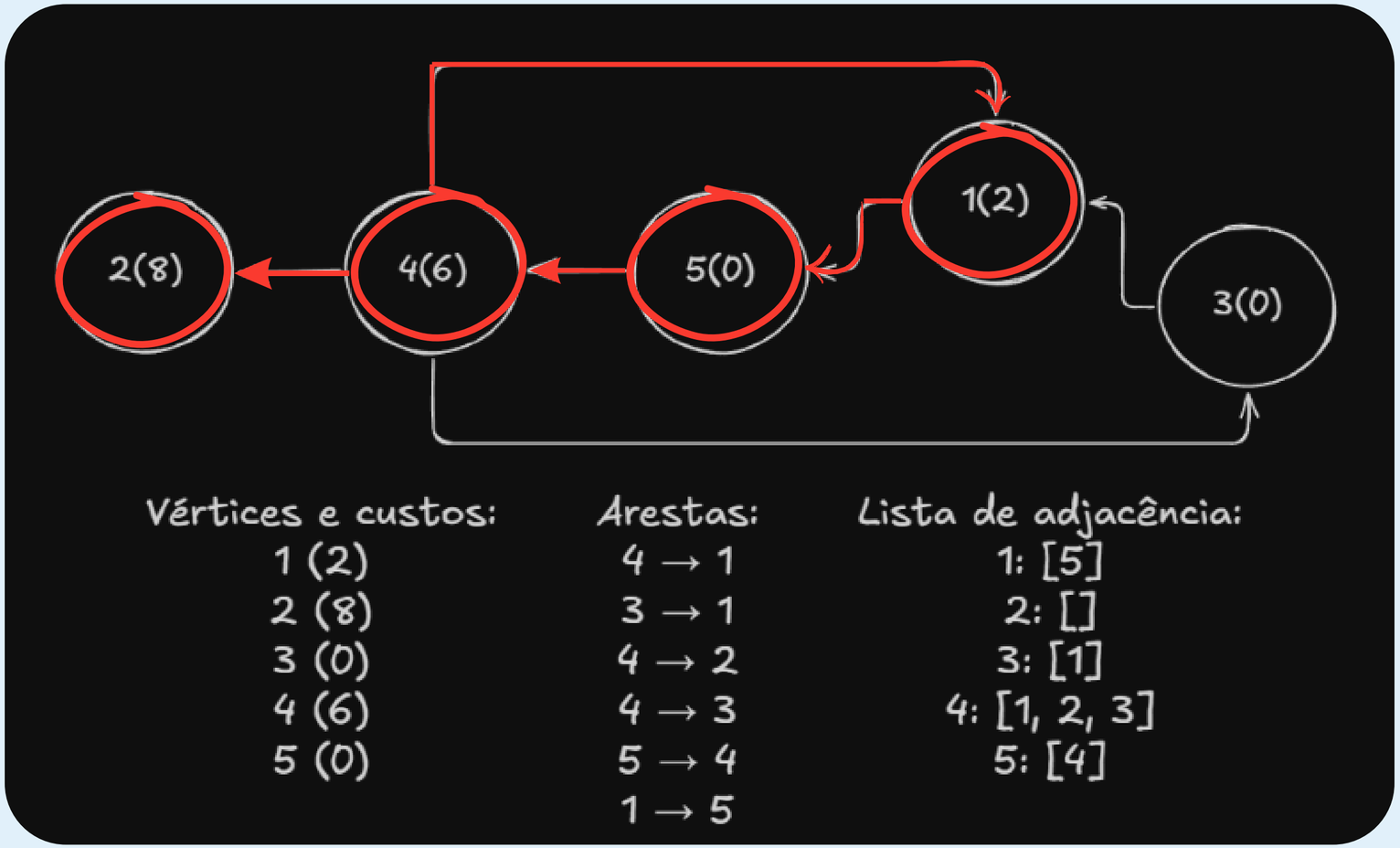
EdgeTo[ ]:
1: -
2: 4
3:
4: 5
5: 1

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 2, 4, 5]

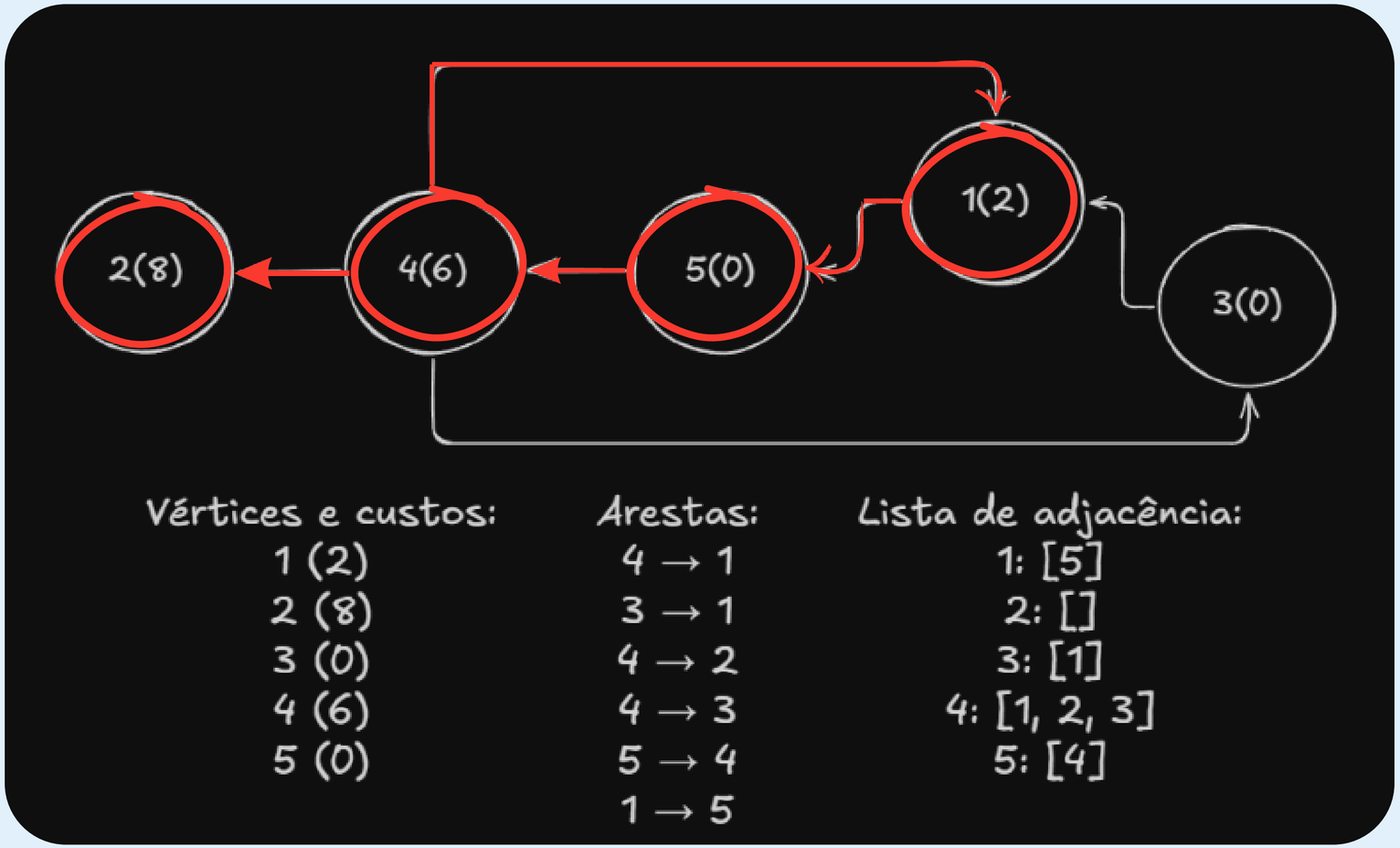
EdgeTo[ ]:
1: -
2: 4
3:
4: 5
5: 1

Pós-ordem:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 2, 4, 5]

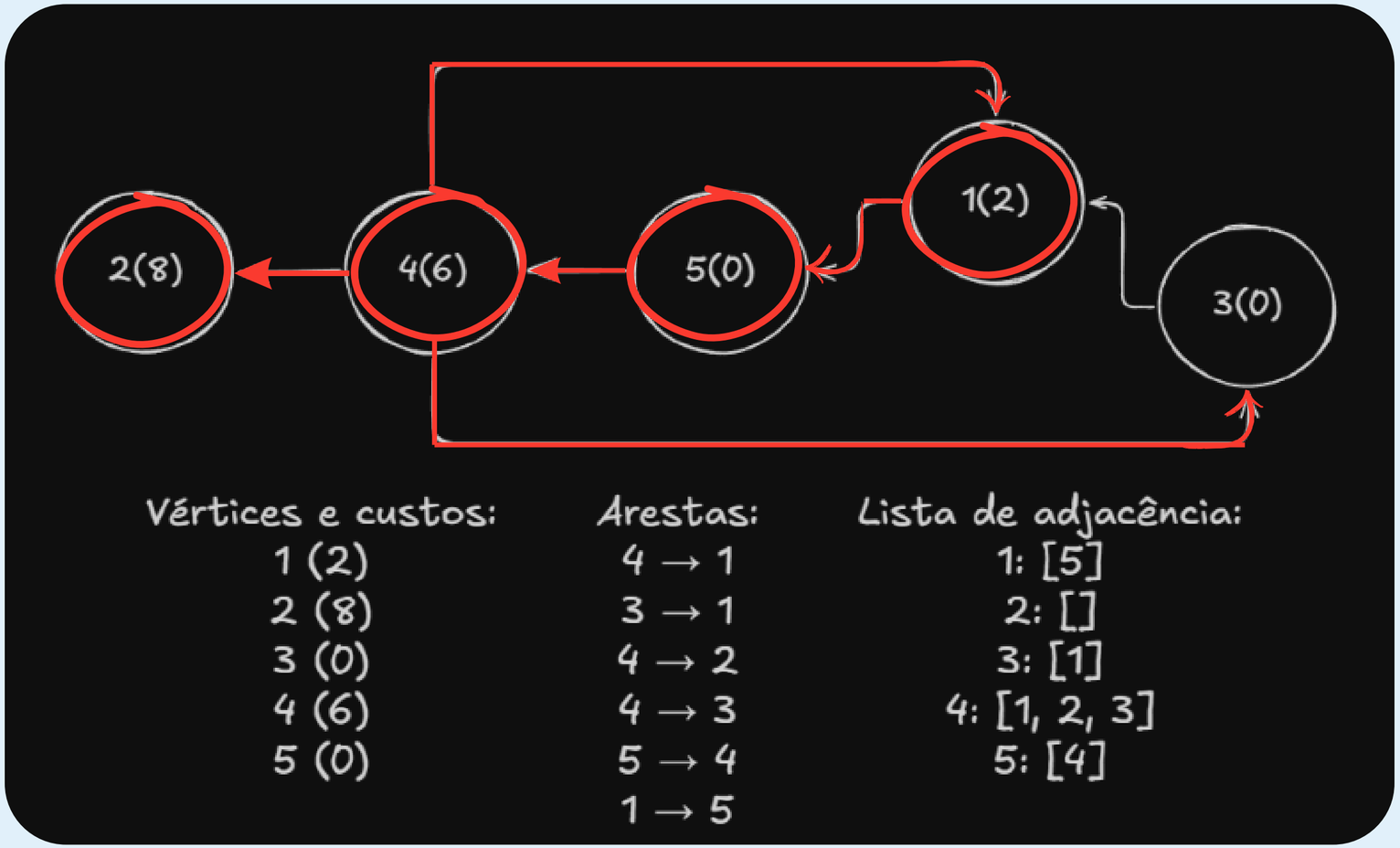
EdgeTo[ ]:
1: -
2: 4
3:
4: 5
5: 1

Pós-ordem:
[2]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 2, 4, 5]

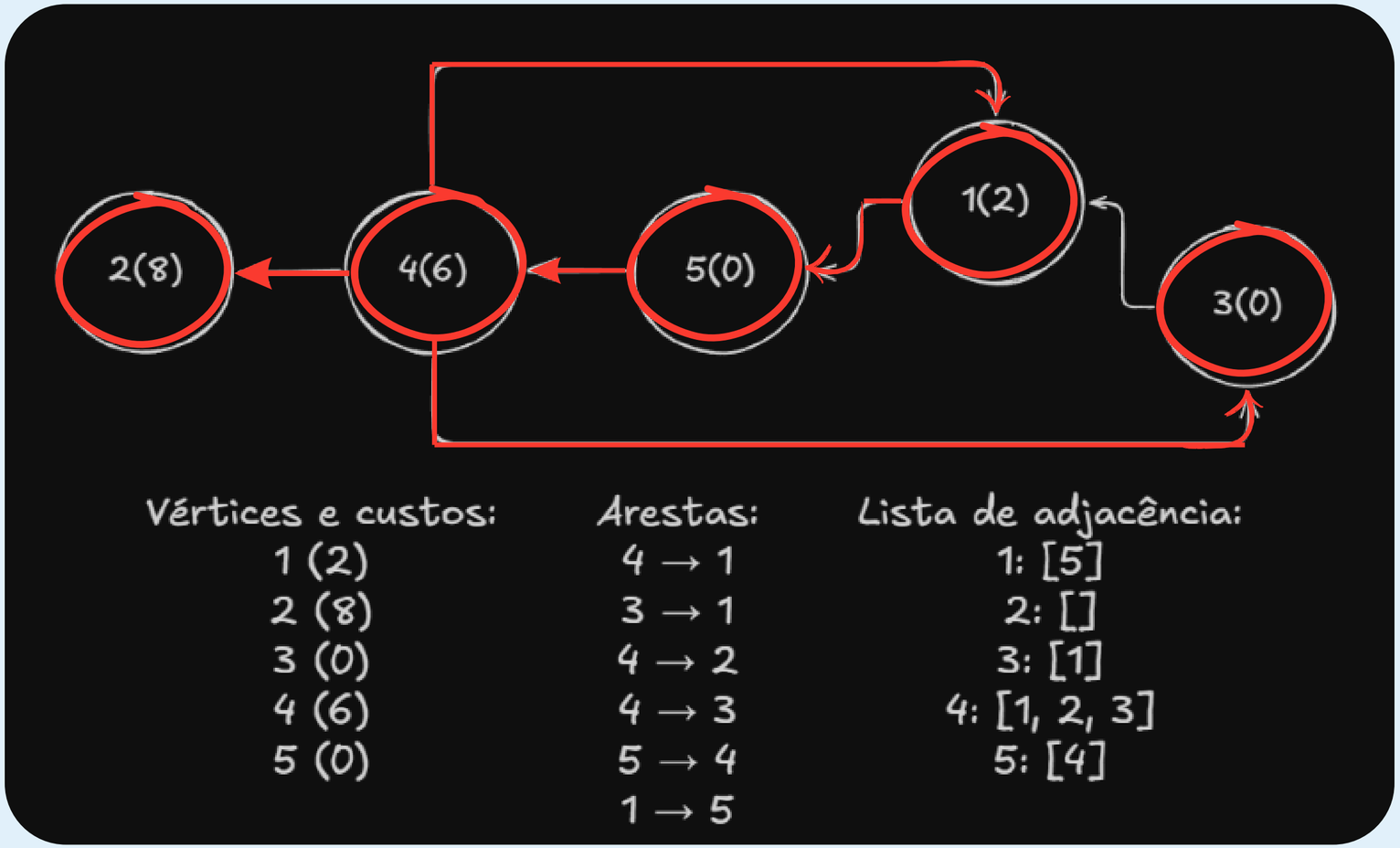
EdgeTo[ ]:
1: -
2: 4
3: 4
4: 5
5: 1

Pós-ordem:
[2]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 2, 3, 4, 5]

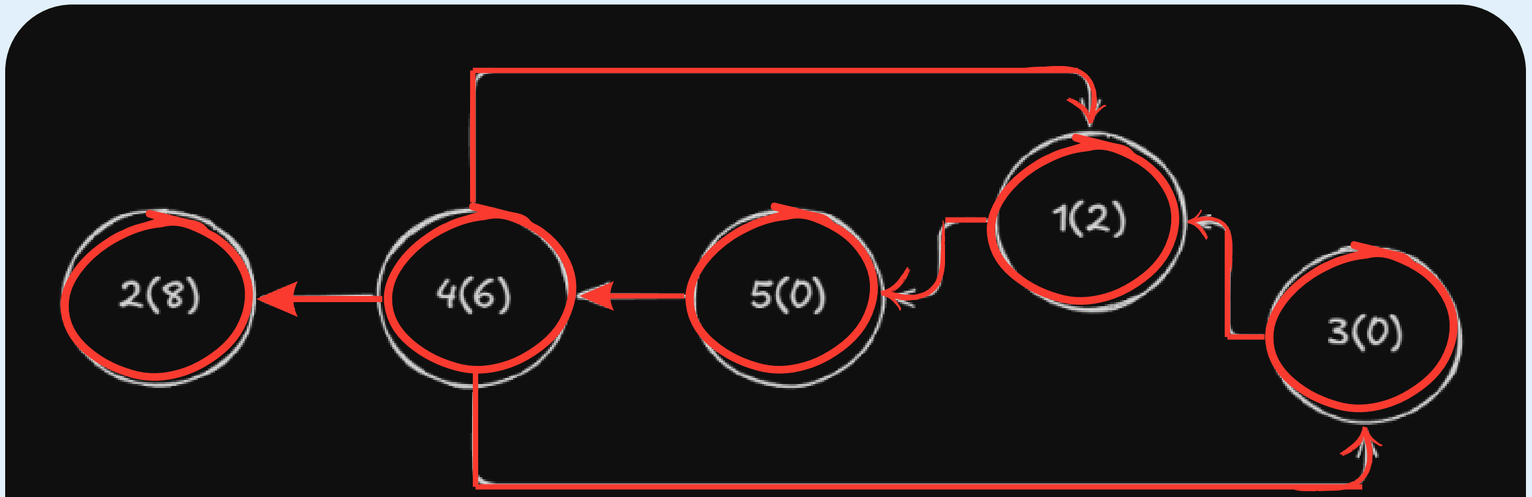
EdgeTo[ ]:
1: -
2: 4
3: 4
4: 5
5: 1

Pós-ordem:
[2]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Vértices e custos:
1 (2)
2 (8)
3 (0)
4 (6)
5 (0)

Arestas:
4 → 1
3 → 1
4 → 2
4 → 3
5 → 4
1 → 5

Lista de adjacência:
1: [5]
2: []
3: [1]
4: [1, 2, 3]
5: [4]

Marked:
[1, 2, 3, 4, 5]

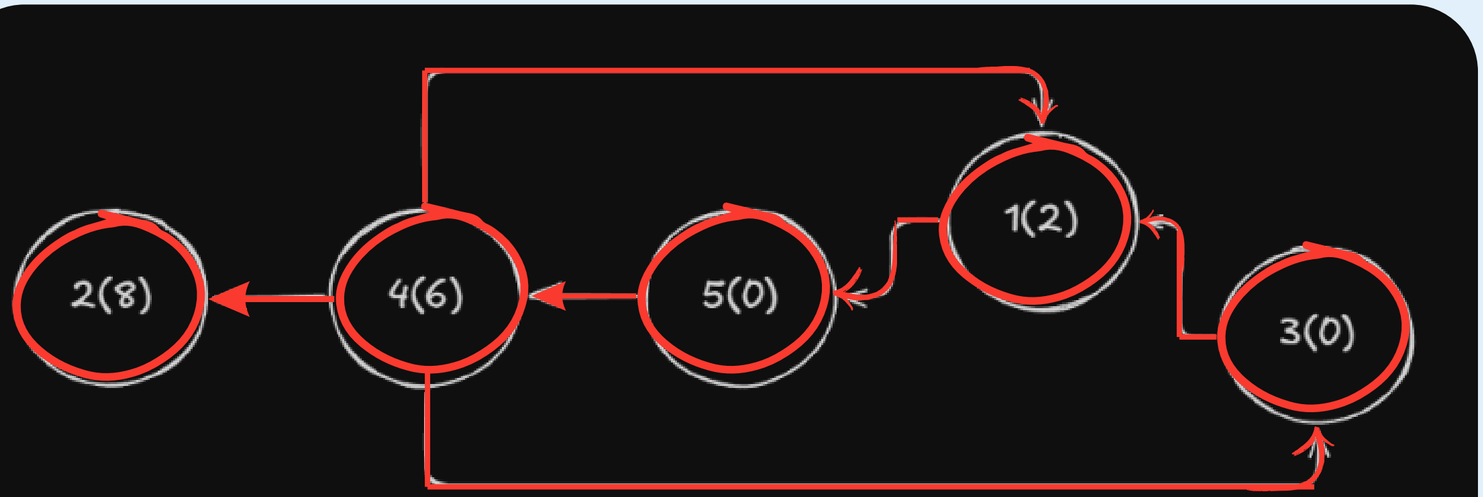
EdgeTo[ ]:
1: -
2: 4
3: 4
4: 5
5: 1

Pós-ordem:
[2]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Vértices e custos:
1 (2)
2 (8)
3 (0)
4 (6)
5 (0)

Arestas:
4 → 1
3 → 1
4 → 2
4 → 3
5 → 4
1 → 5

Lista de adjacência:
1: [5]
2: []
3: [1]
4: [1, 2, 3]
5: [4]

Marked:
[1, 2, 3, 4, 5]

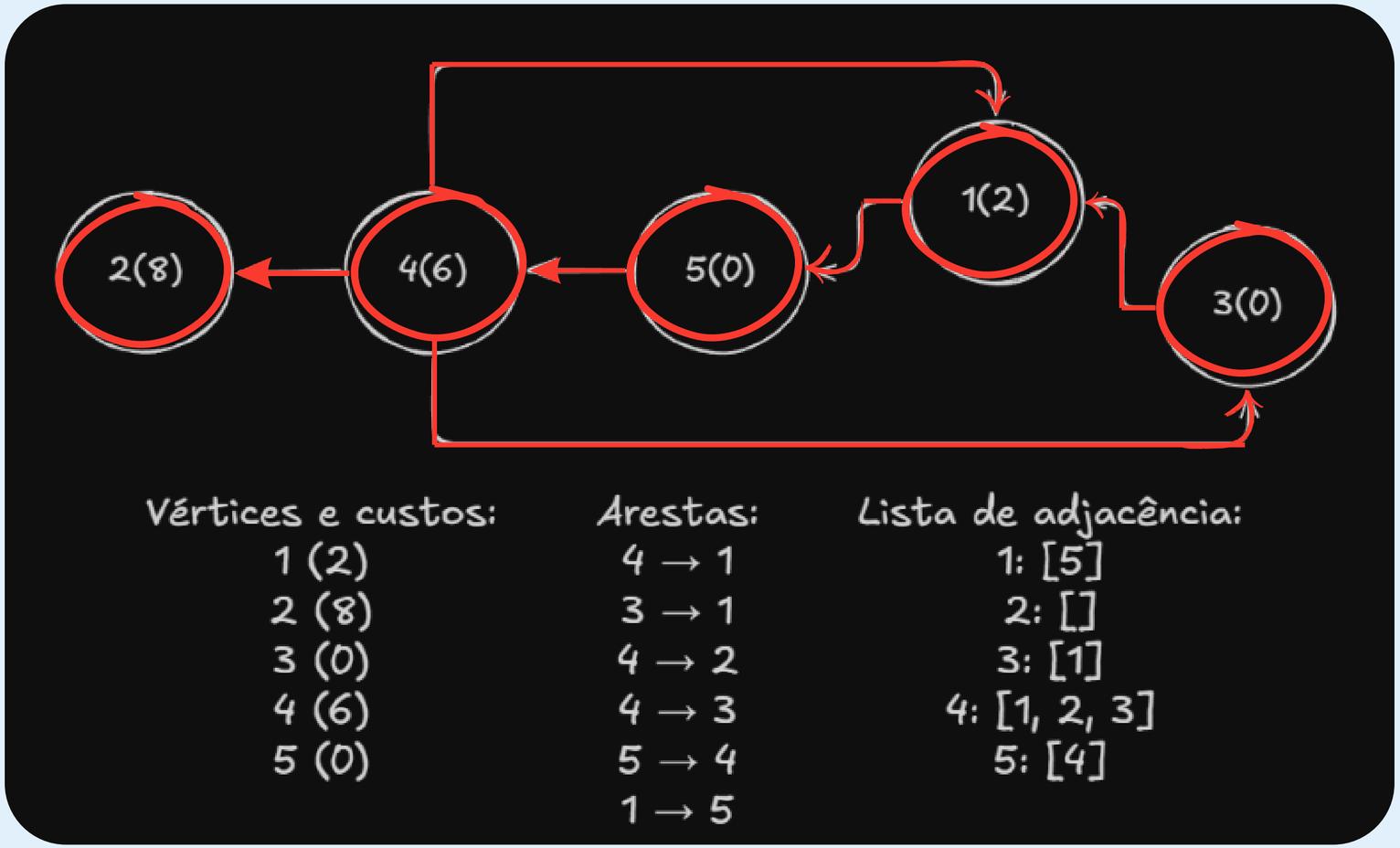
EdgeTo[ ]:
1: -
2: 4
3: 4
4: 5
5: 1

Pós-ordem:
[2, 3]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 2, 3, 4, 5]

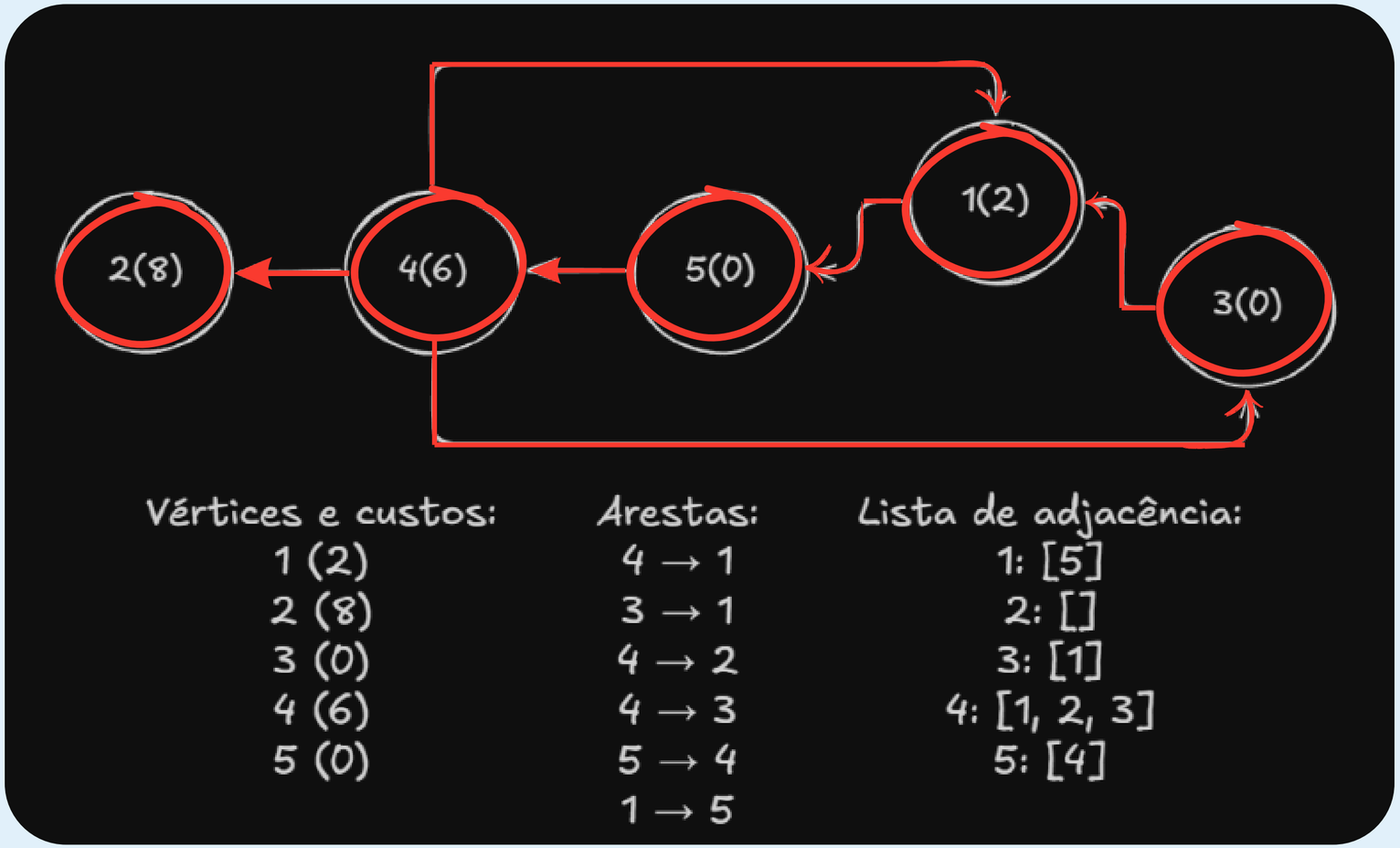
EdgeTo[ ]:
1: -
2: 4
3: 4
4: 5
5: 1

Pós-ordem:
[2, 3, 4]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 2, 3, 4, 5]

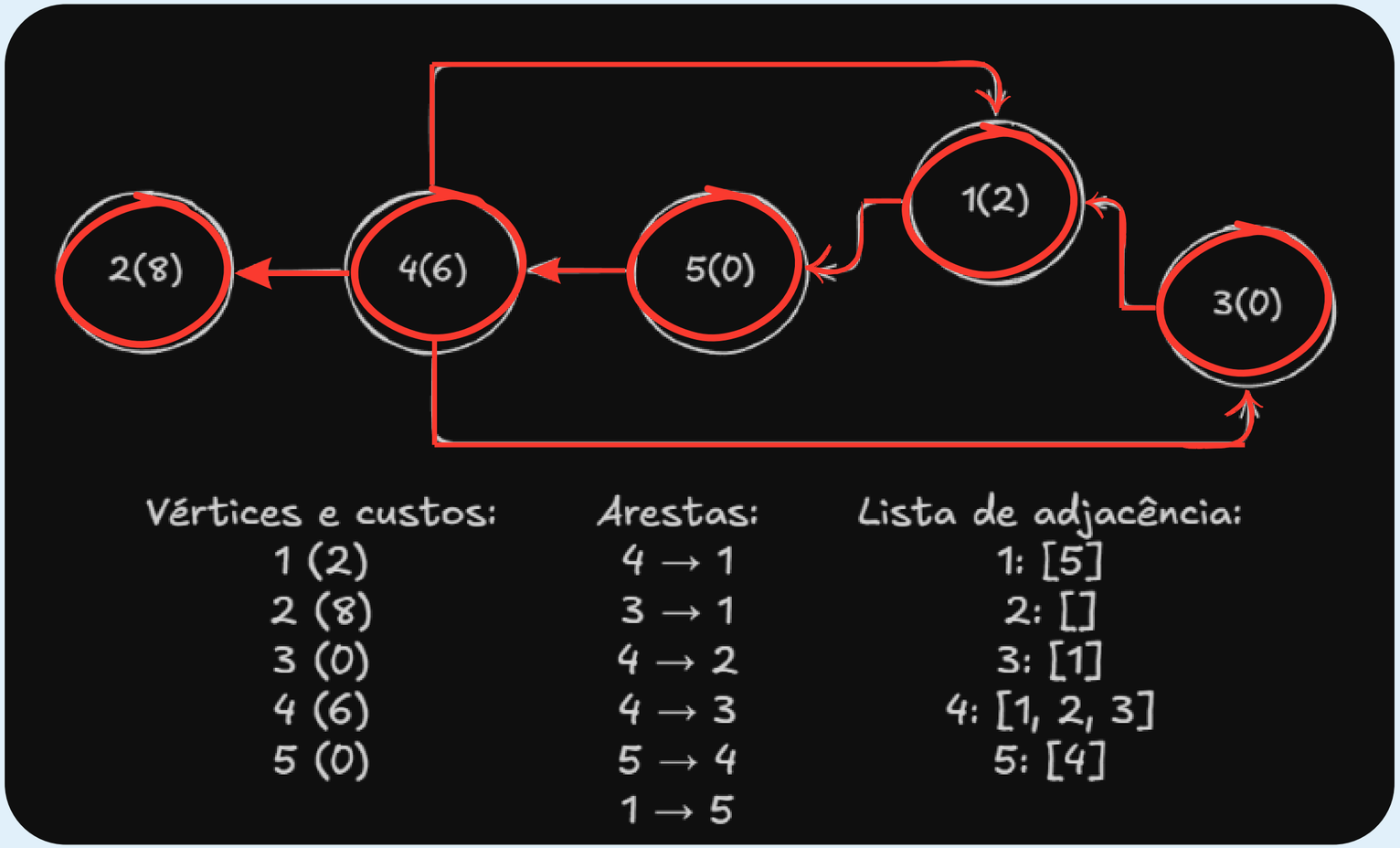
EdgeTo[ ]:
1: -
2: 4
3: 4
4: 5
5: 1

Pós-ordem:
[2, 3, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 3: DFS no grafo reverso



Marked:
[1, 2, 3, 4, 5]

EdgeTo[ ]:
1: -
2: 4
3: 4
4: 5
5: 1

Pós-ordem:
[2, 3, 4, 5, 1]

Algoritmo de Kosaraju

Checkposts

Passo 4: Pós-ordem reversa

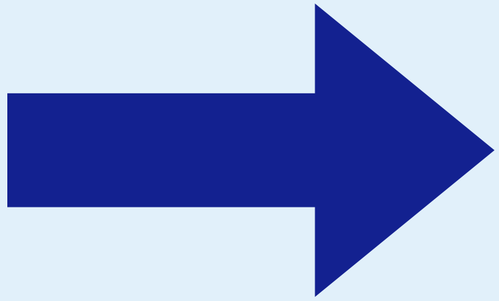
Pós-ordem:
[2, 3, 4, 5, 1]

Algoritmo de Kosaraju

Checkposts

Passo 4: Pós-ordem reversa

Pós-ordem:
[2, 3, 4, 5, 1]

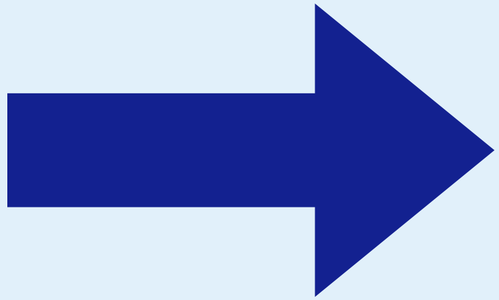


Algoritmo de Kosaraju

Checkposts

Passo 4: Pós-ordem reversa

Pós-ordem:
[2, 3, 4, 5, 1]

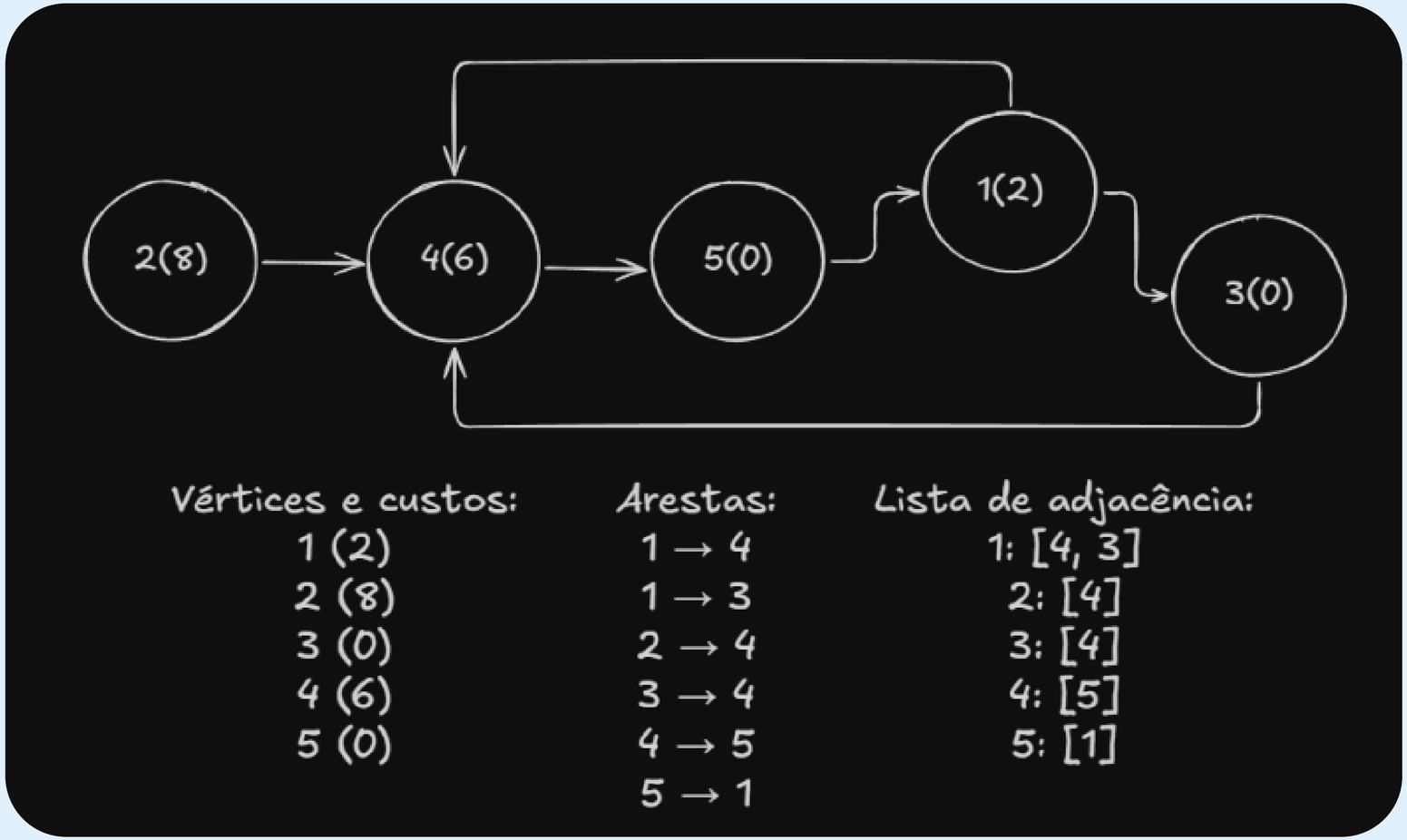


Pós-ordem reversa:
[1, 5, 4, 3, 2]

Algoritmo de Kosaraju

Checkposts

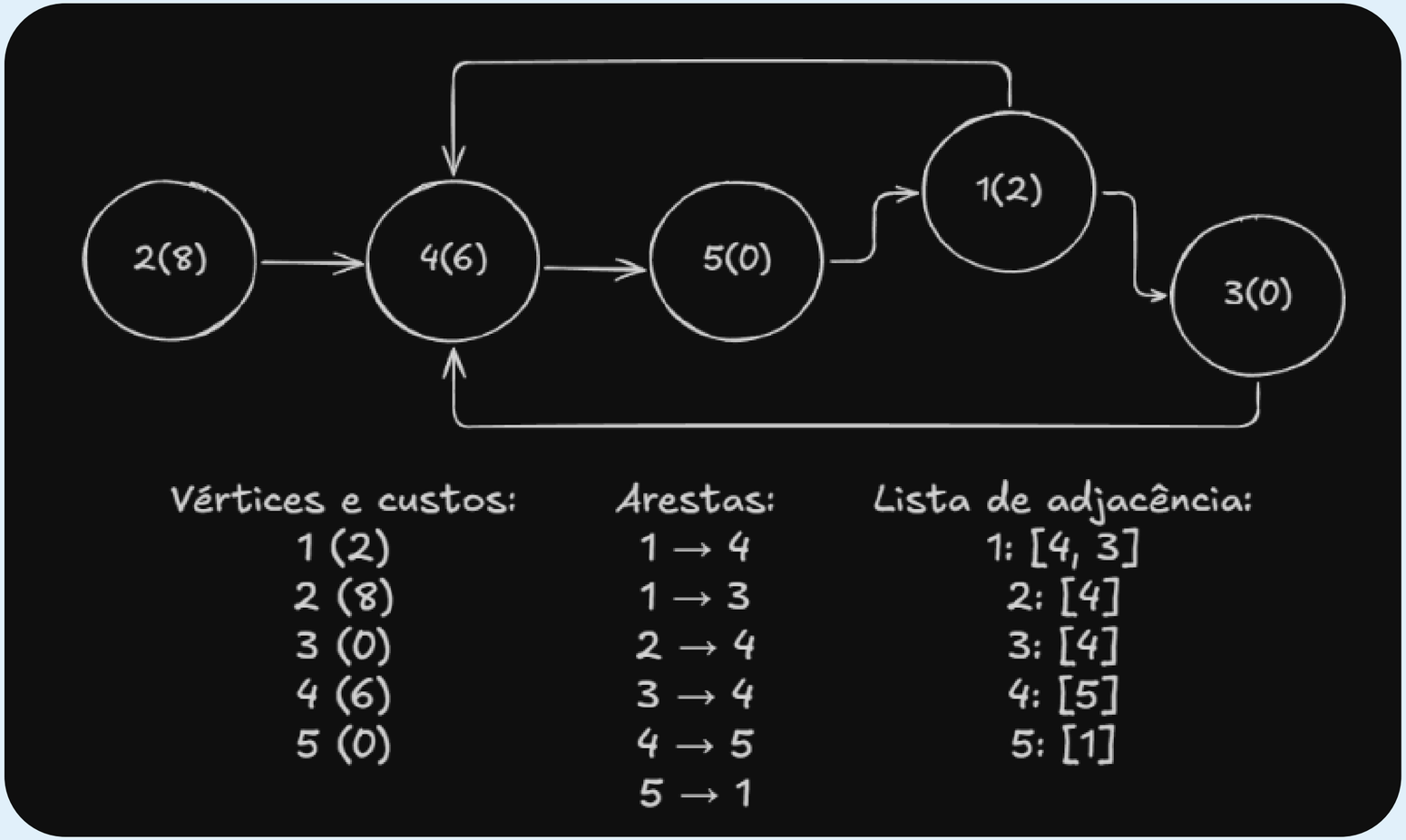
Passo 5: DFS no grafo original



Algoritmo de Kosaraju

Checkposts

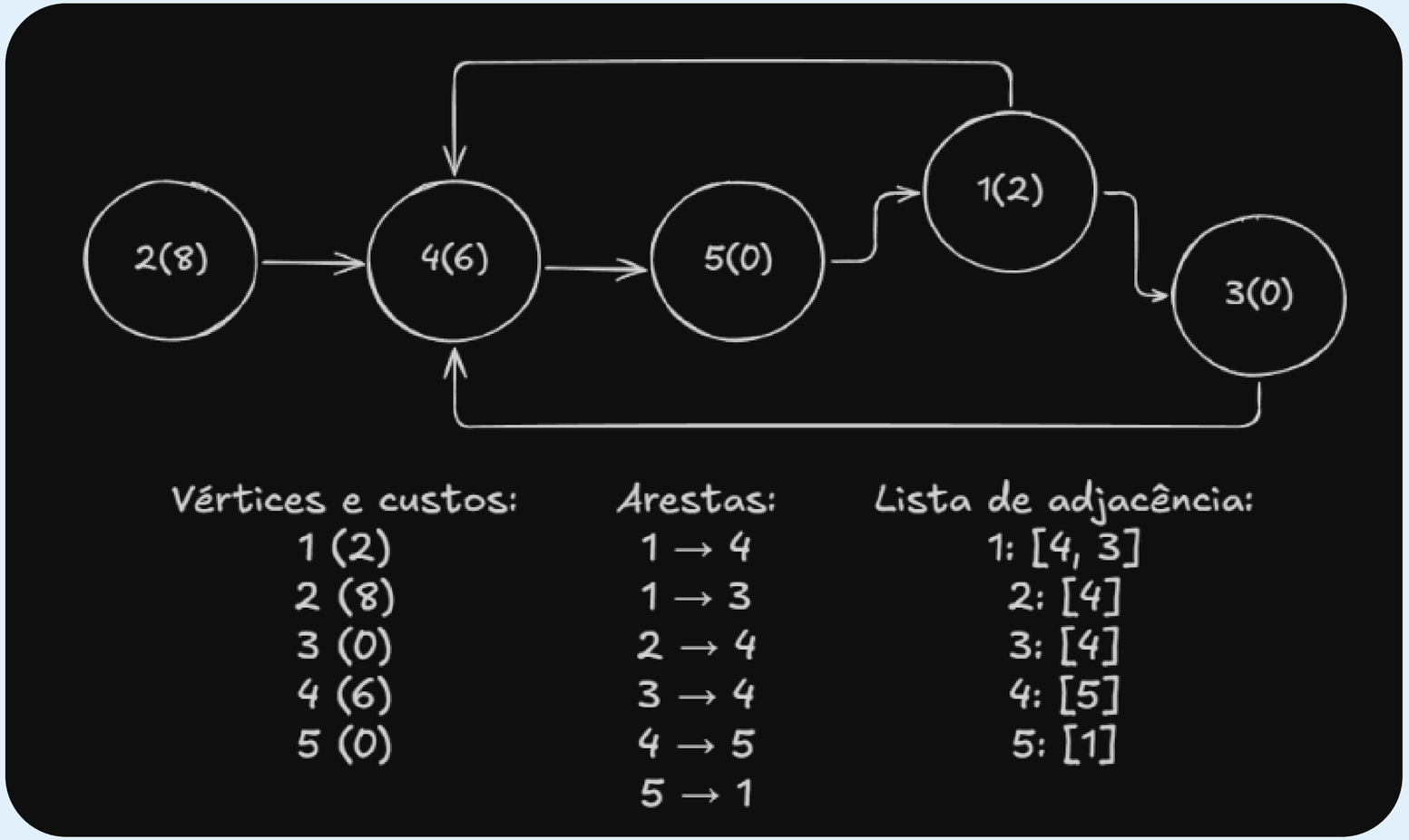
Passo 5: DFS no grafo original



Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



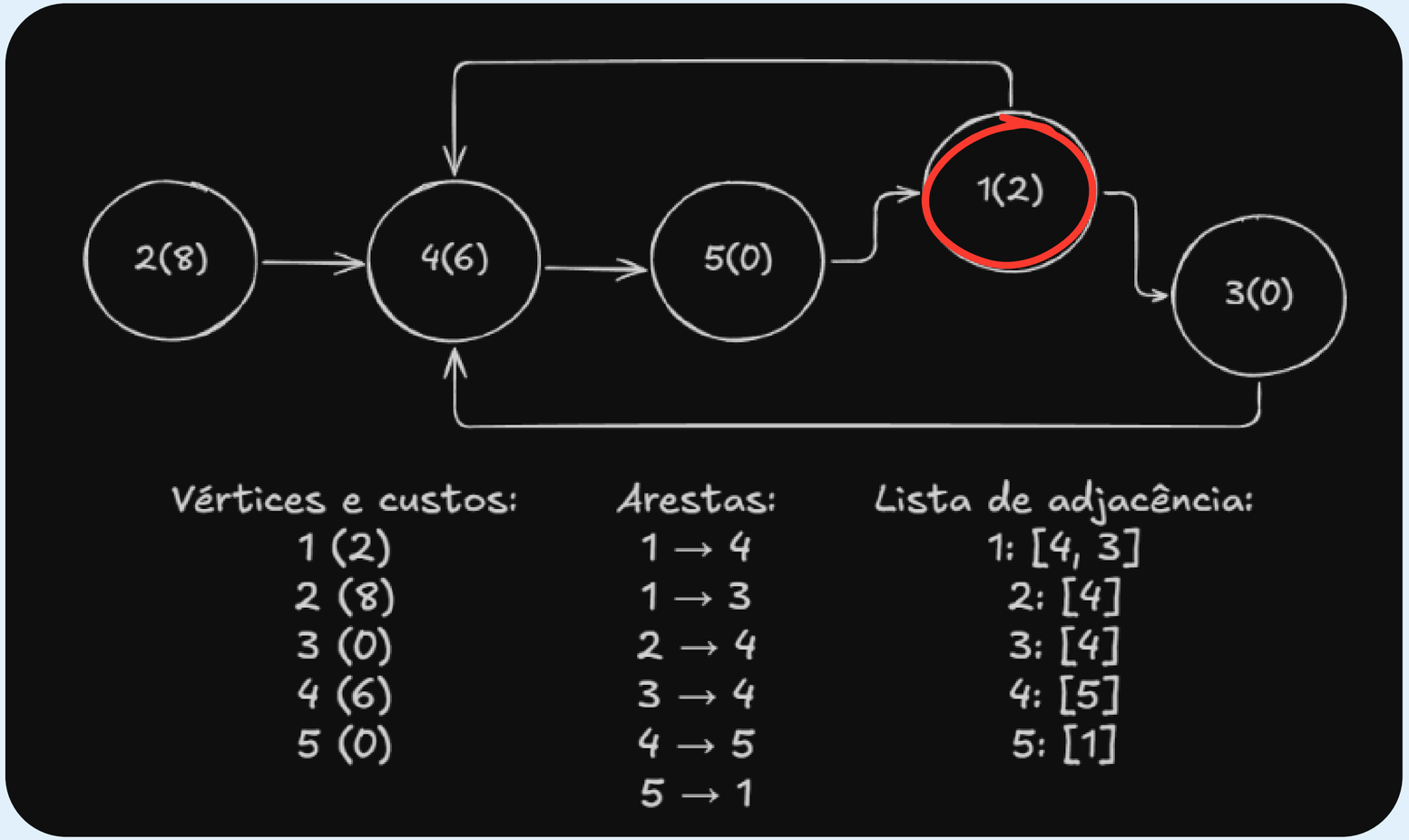
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[ ]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



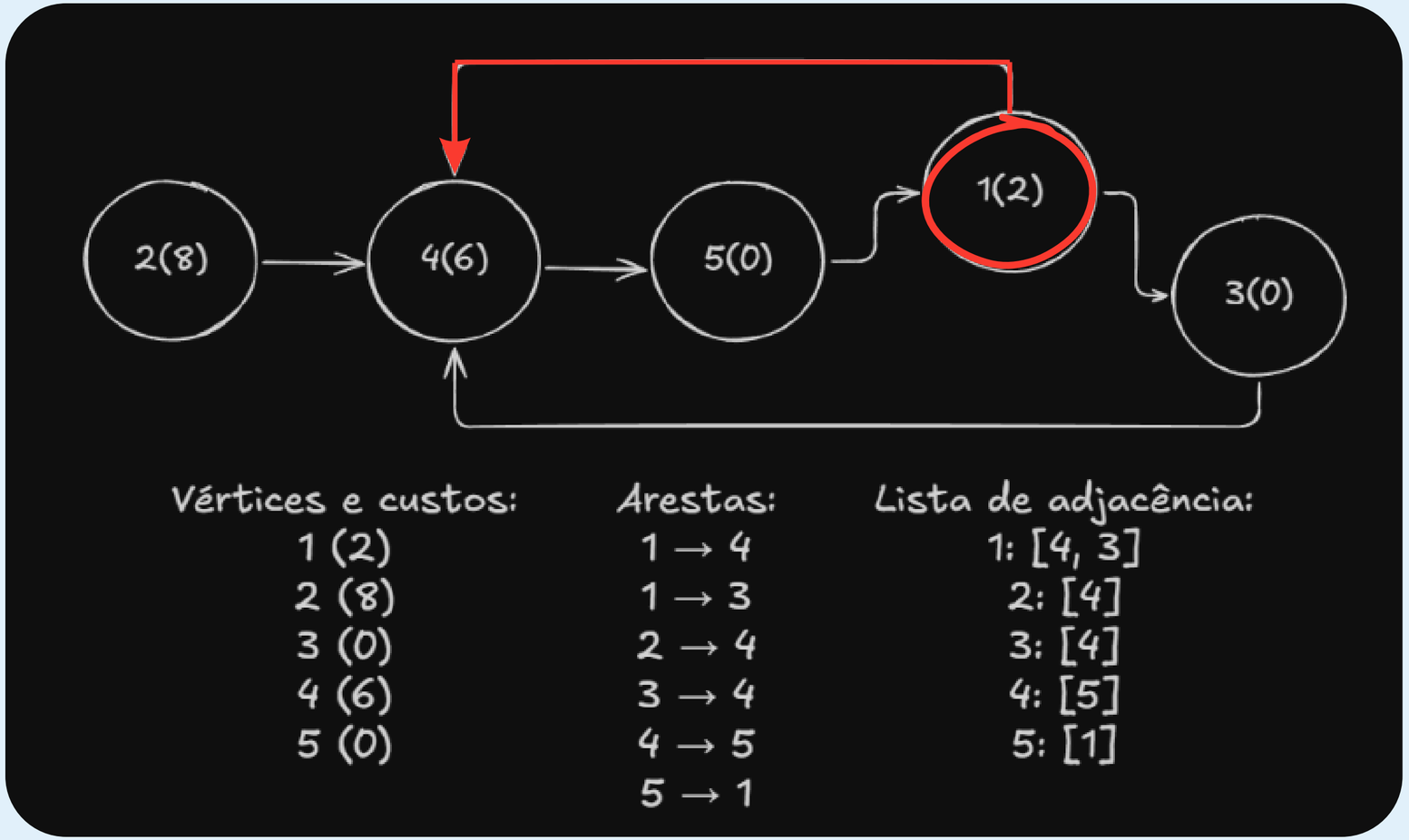
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



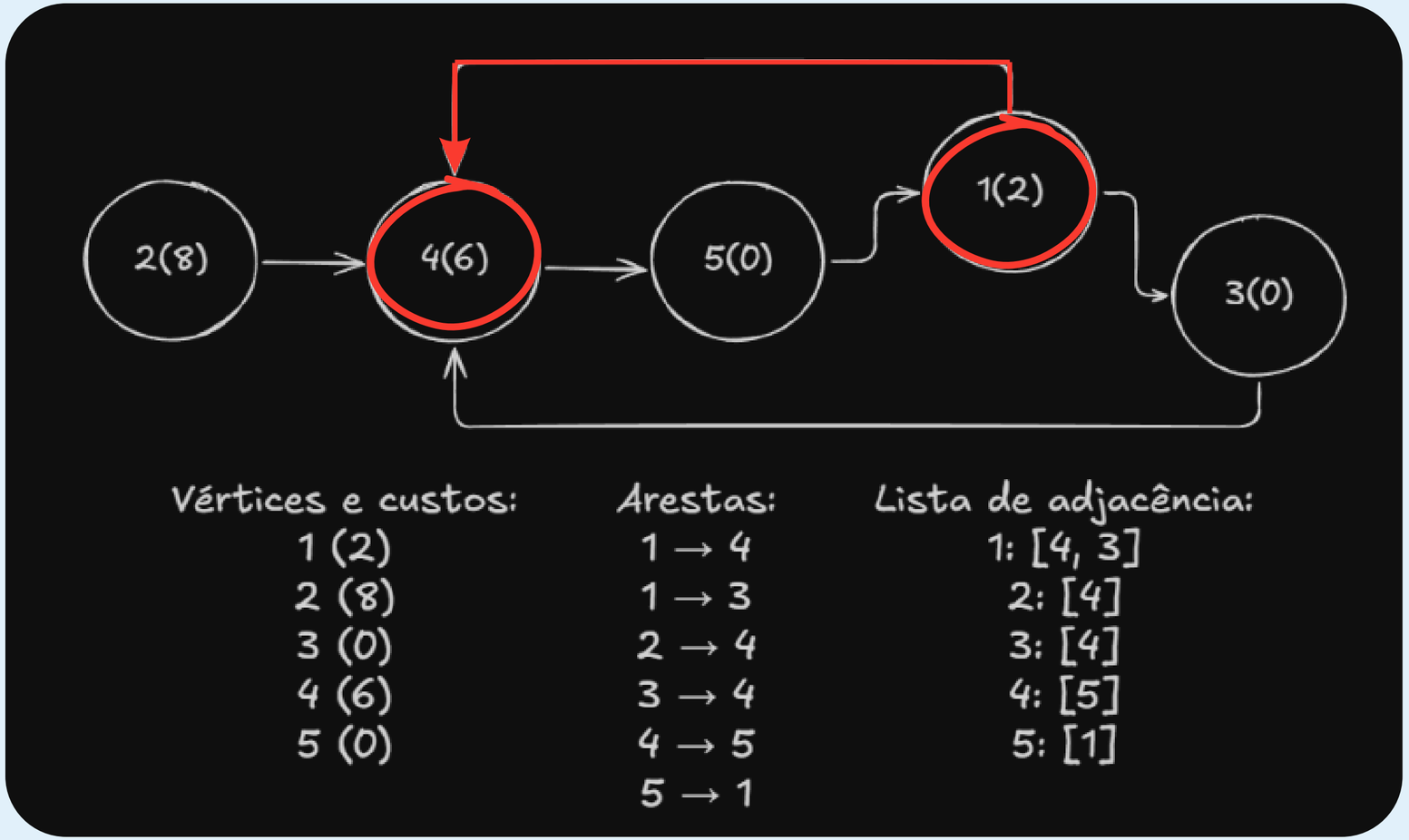
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



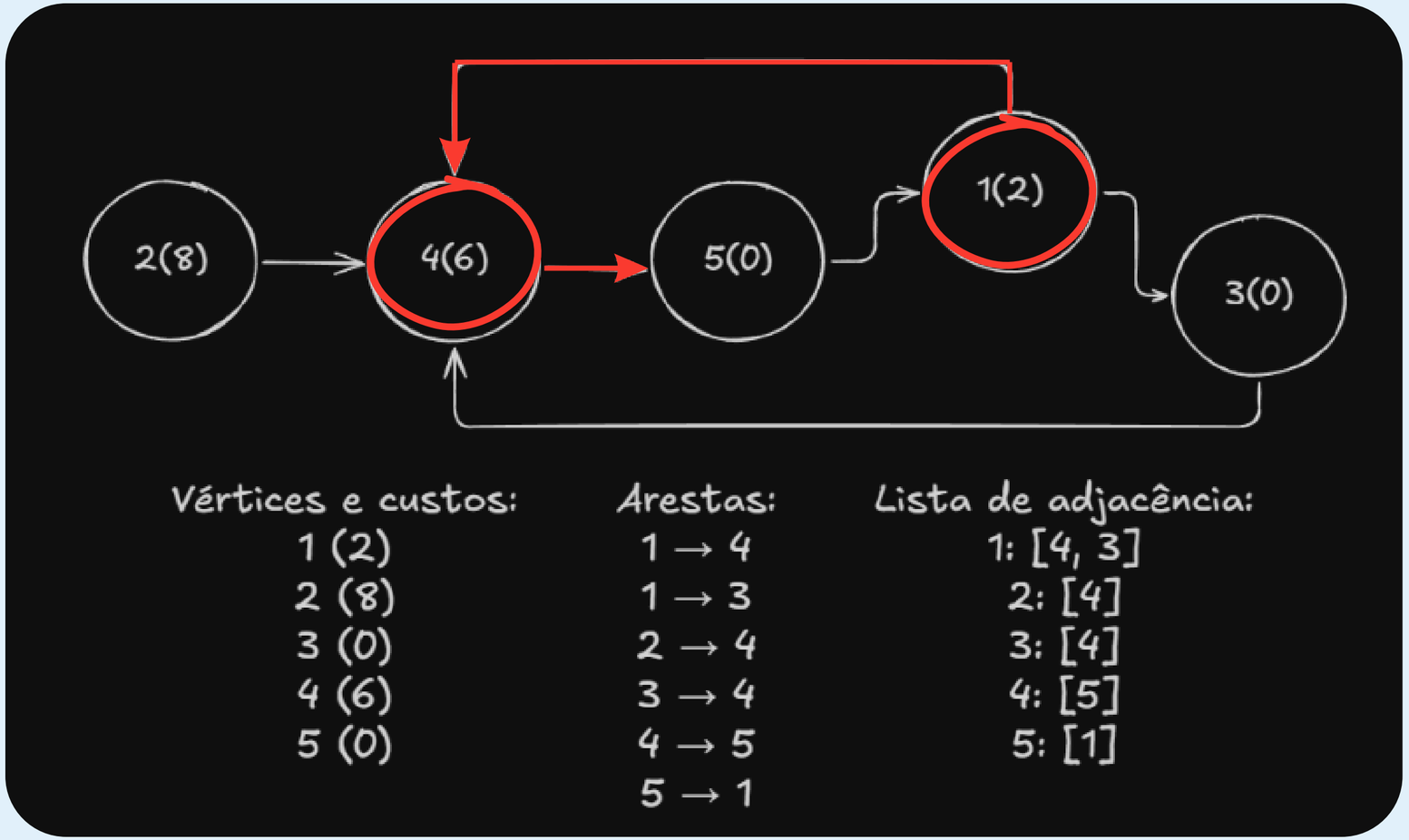
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 4]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



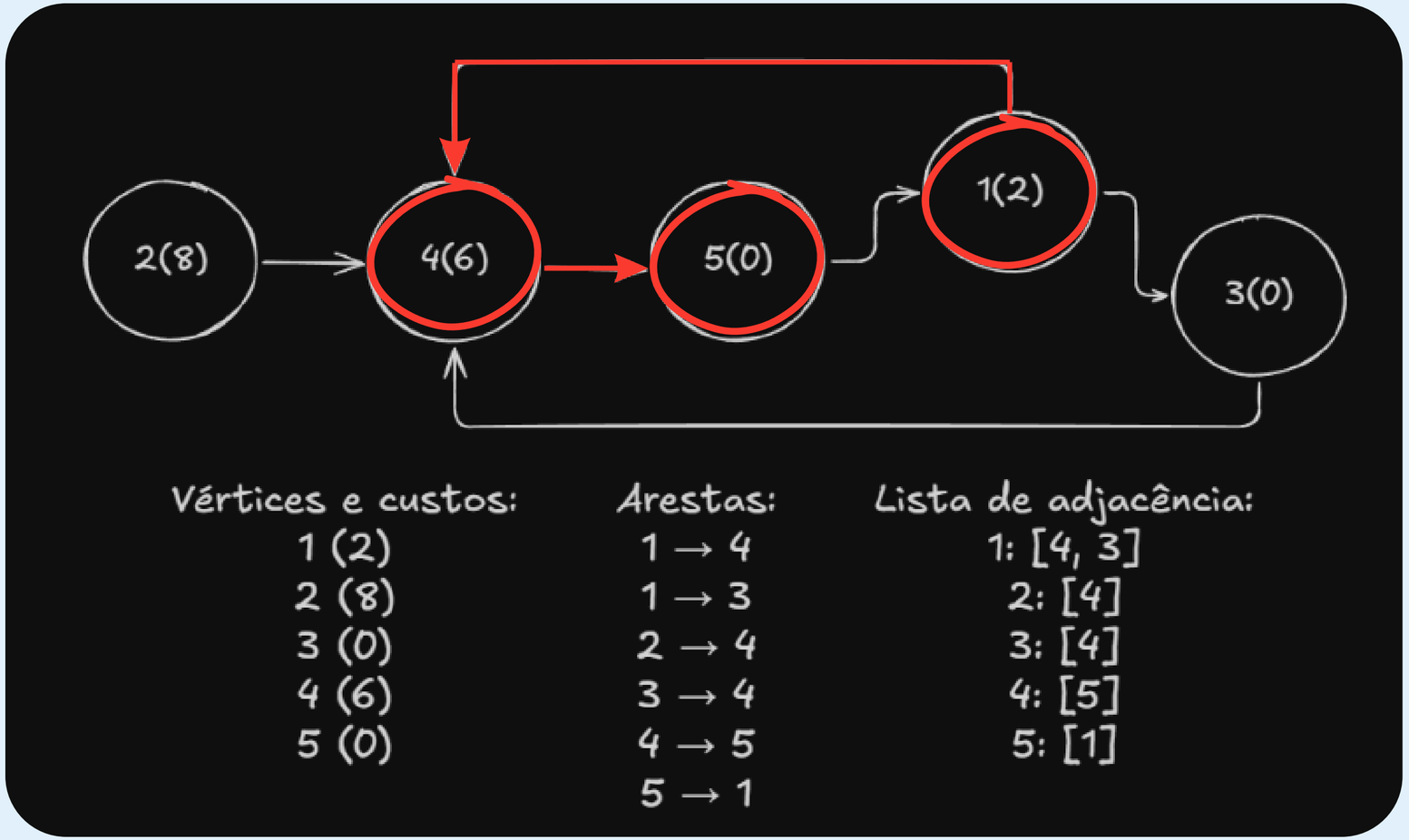
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 4]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



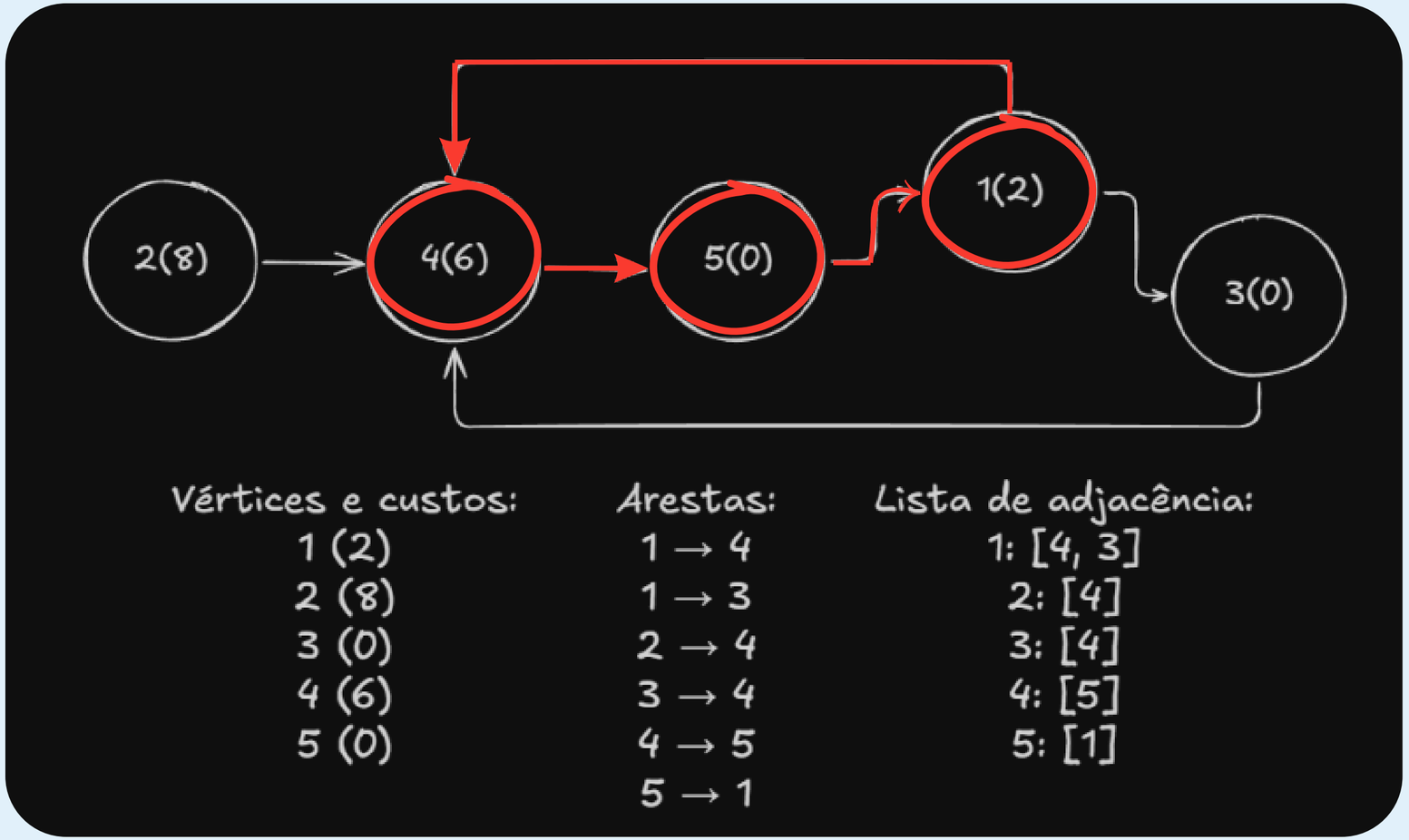
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



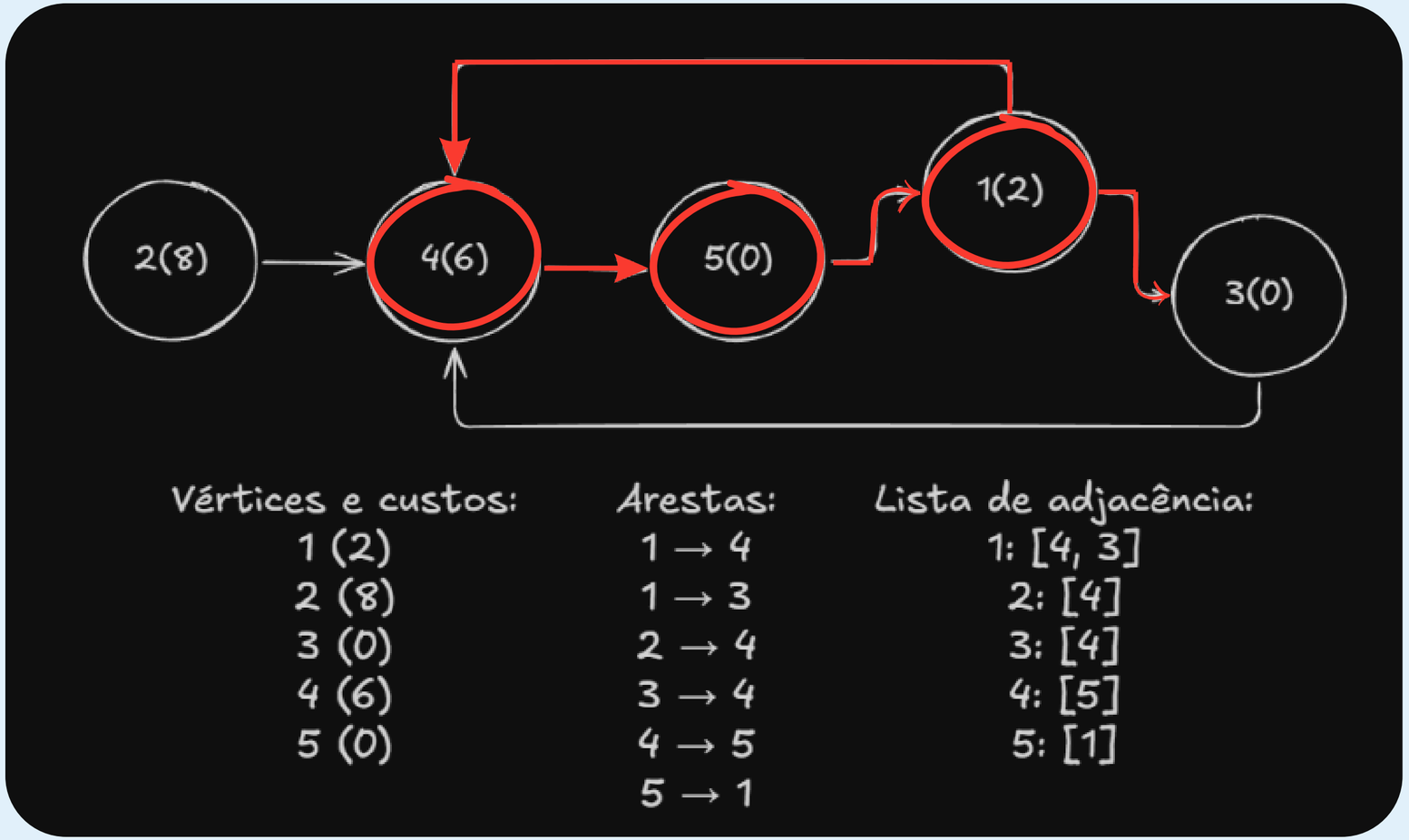
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



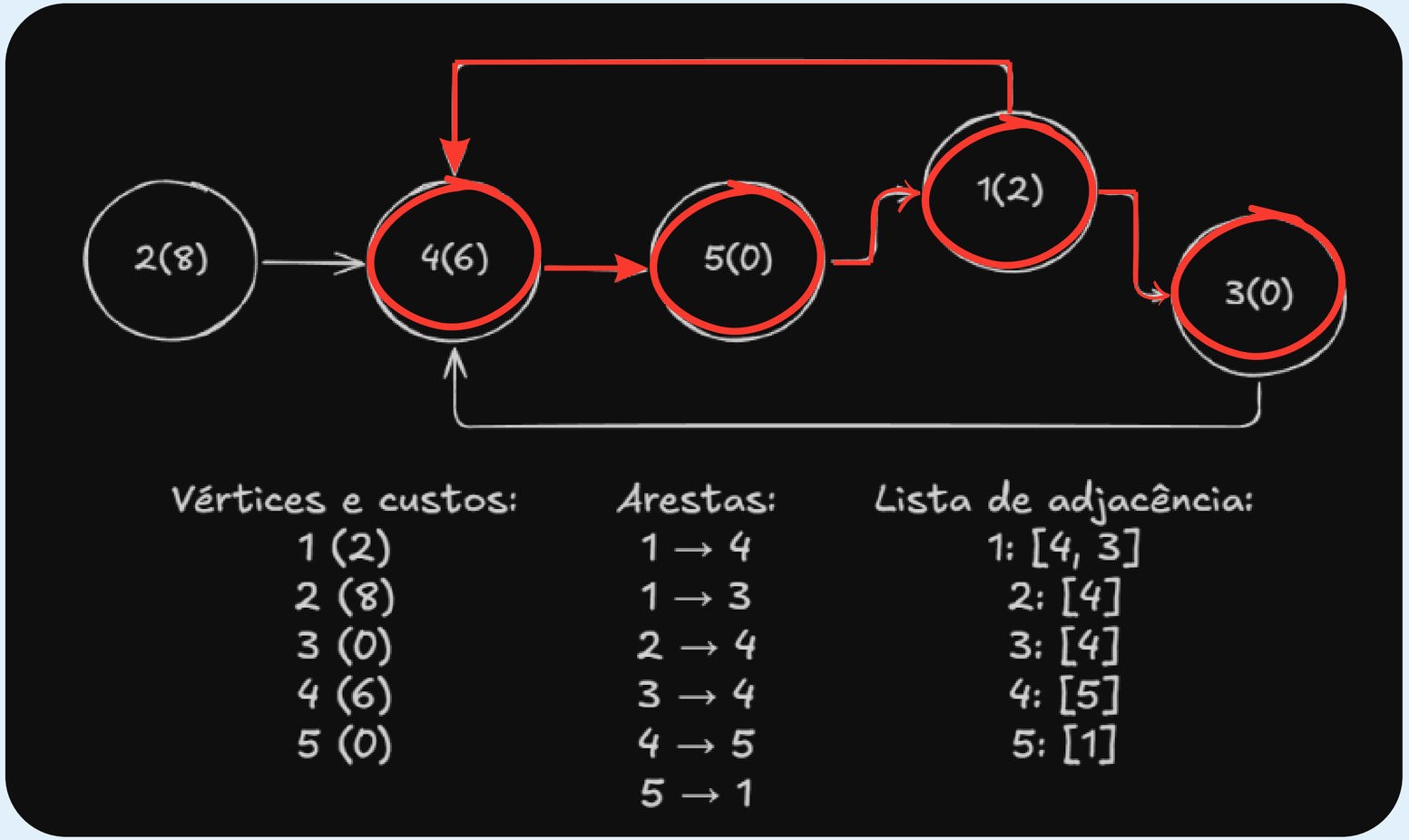
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



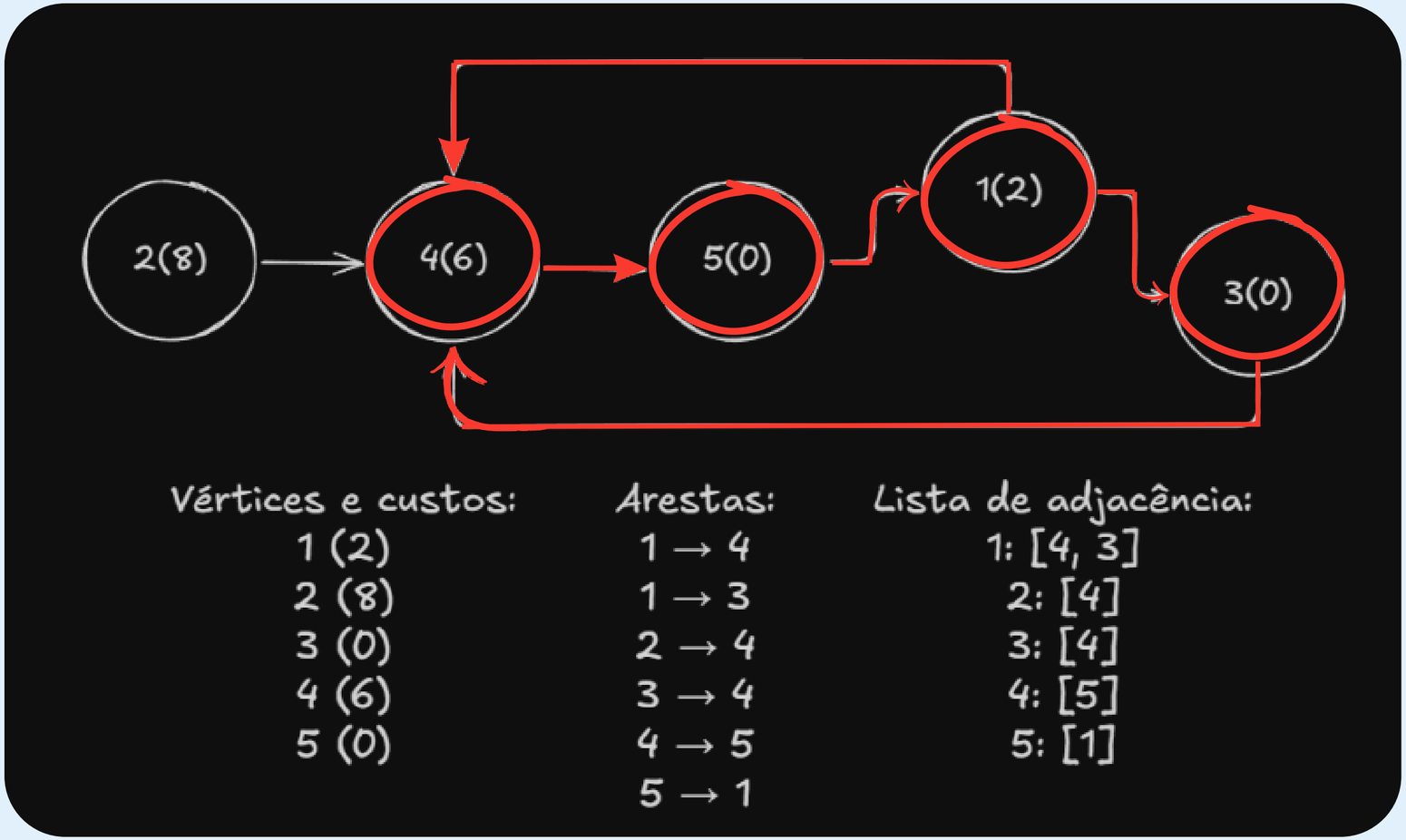
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 3, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



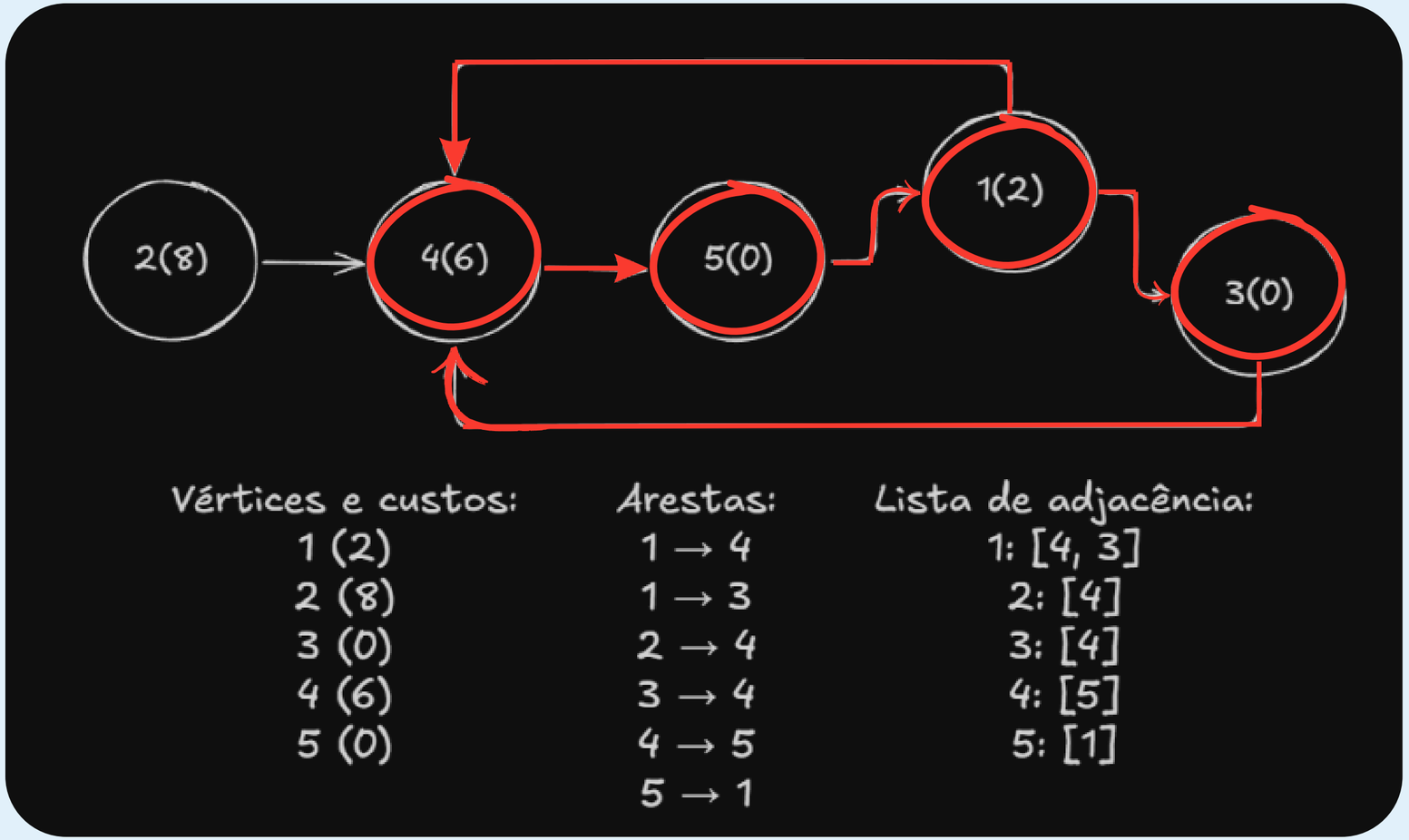
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 3, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



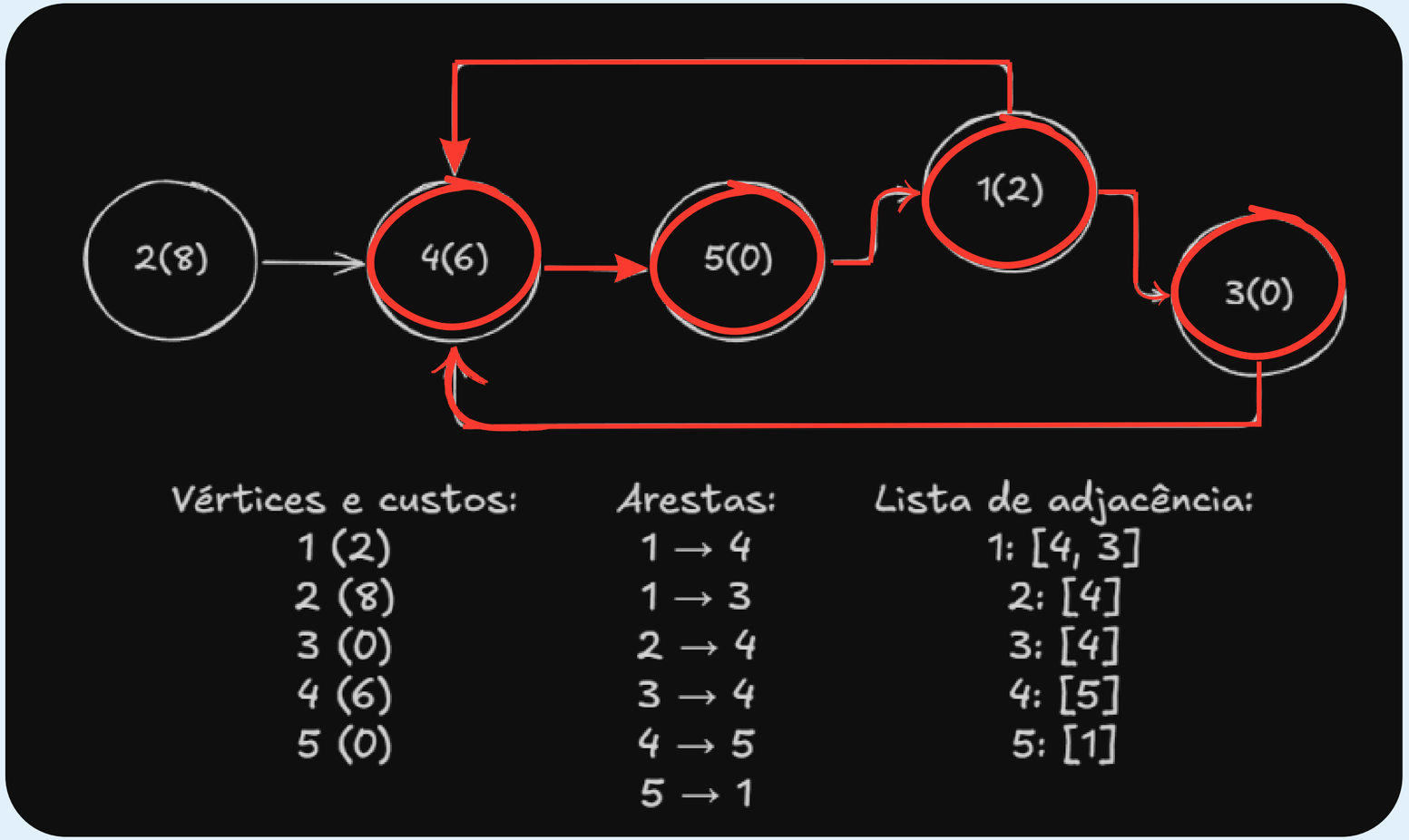
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 3, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



Pós-ordem reversa:
[1, 5, 4, 3, 2]

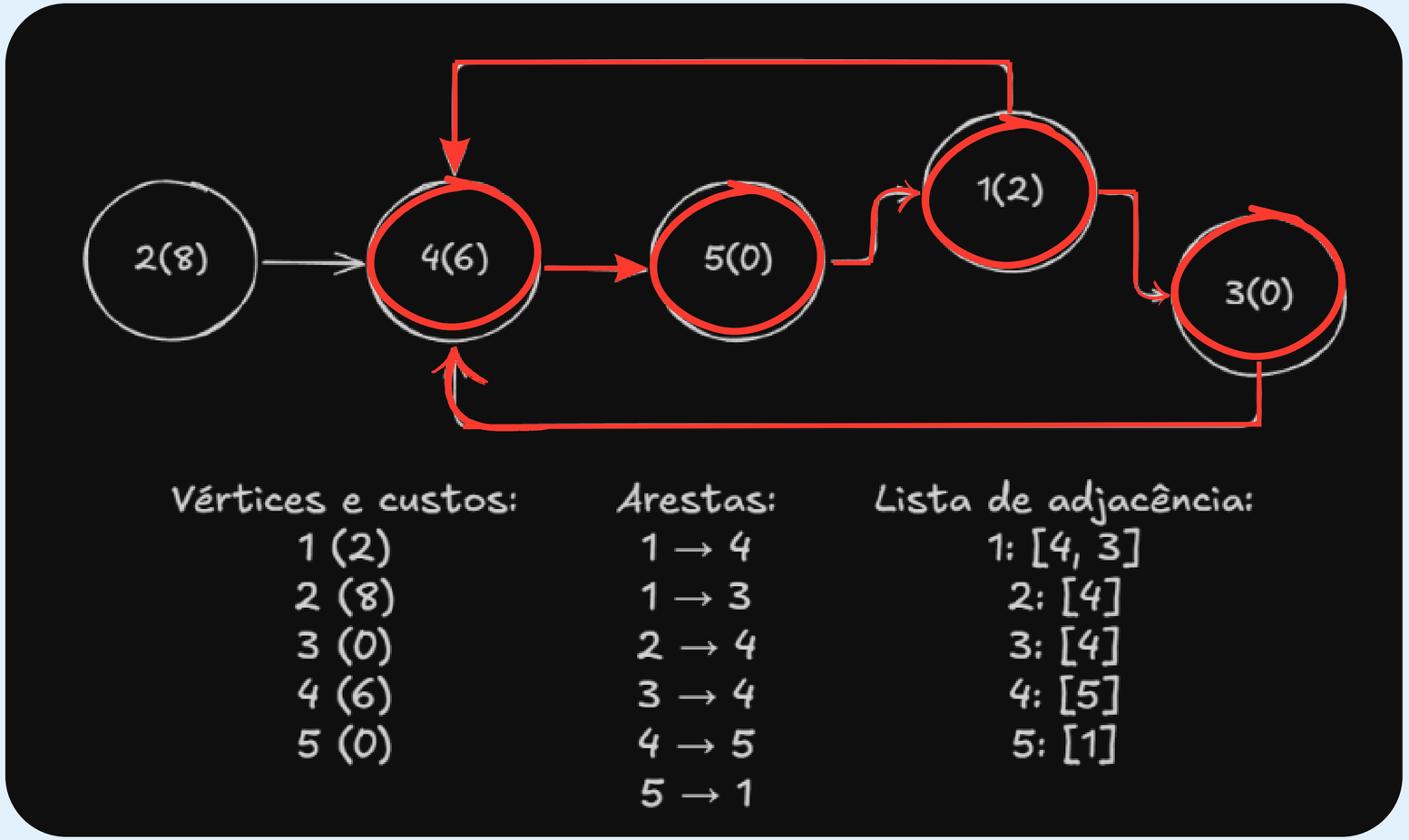
Marked:
[1, 3, 4, 5]

Id = 0

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original



Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 3, 4, 5]

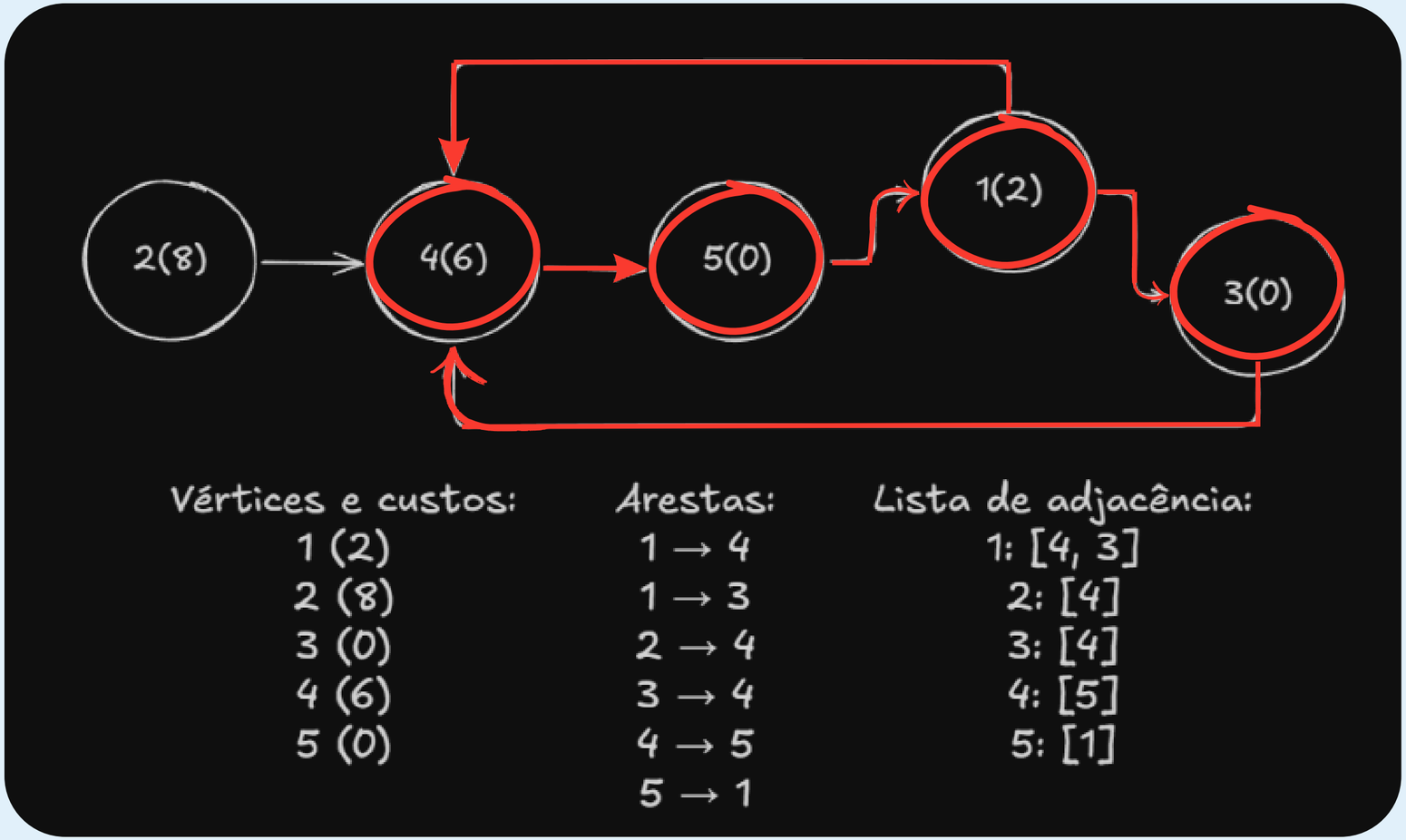
Id = 0

SCC 0 = {1, 3, 4, 5}

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original

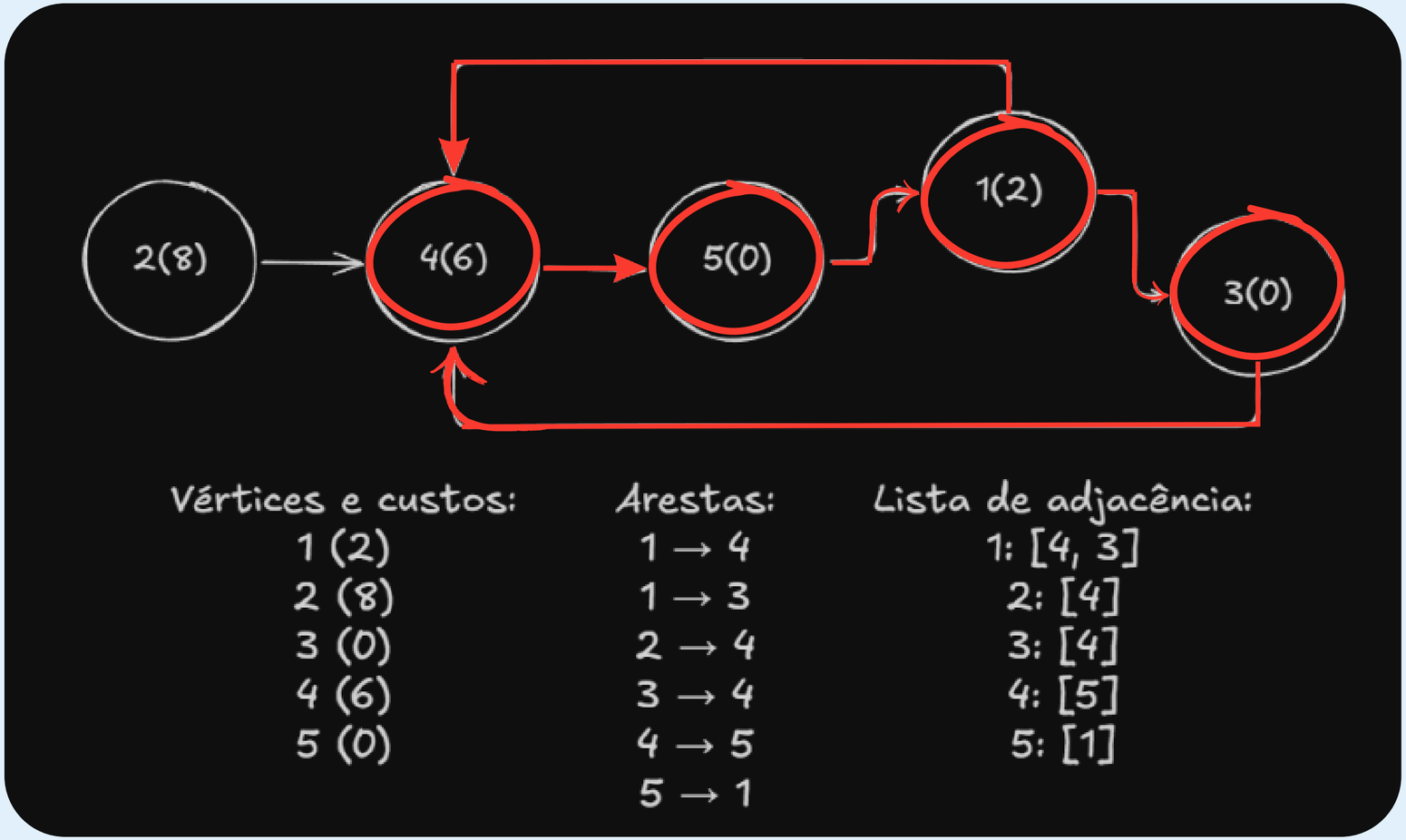


Pós-ordem reversa:
[1, 5, 4, 3, 2]

Algoritmo de Kosaraju

Checkposts

Passo 5: DFS no grafo original

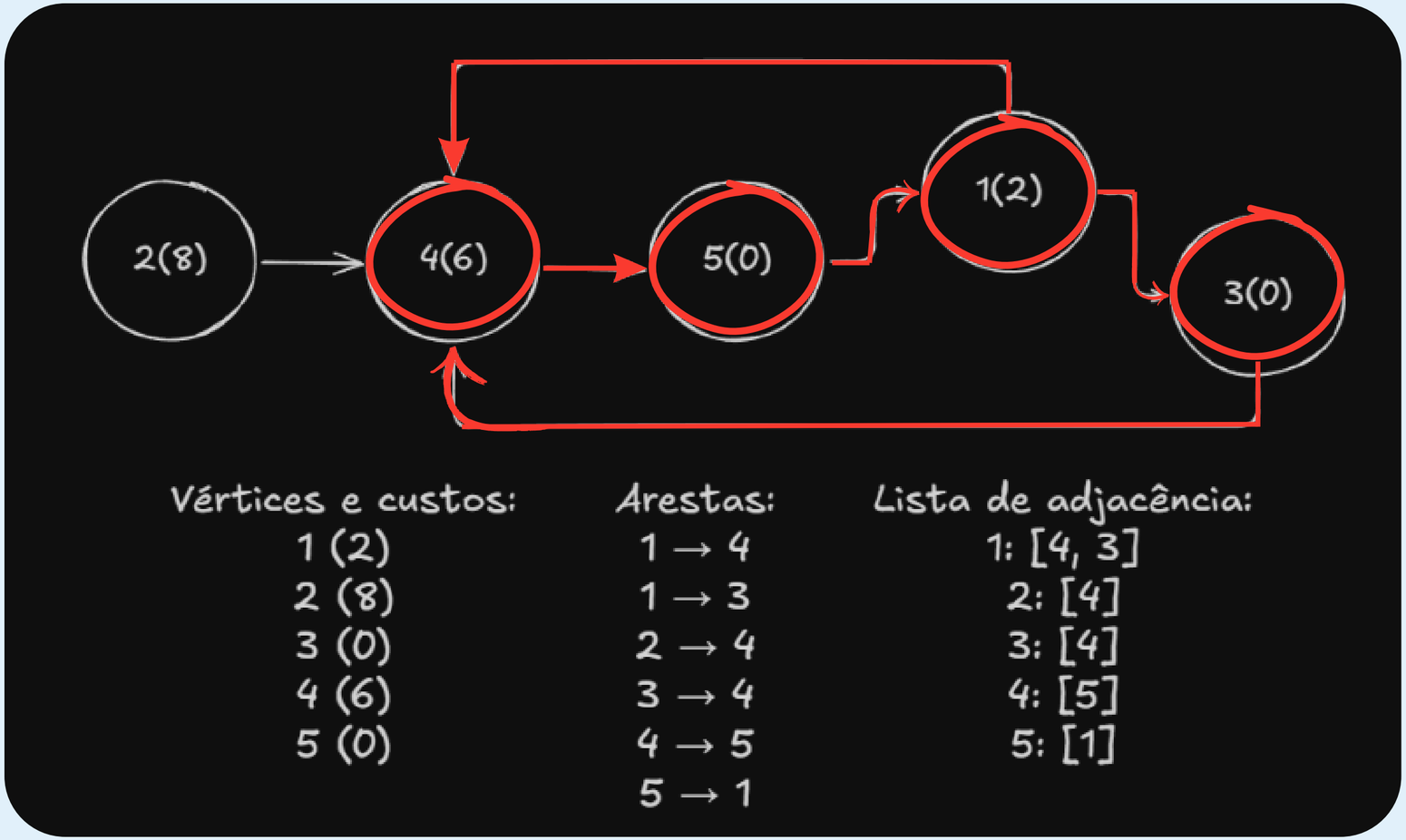


Pós-ordem reversa:
[1, 5, 4, 3, 2]

Algoritmo de Kosaraju

Checkposts

Passo 6: Segunda DFS no grafo original



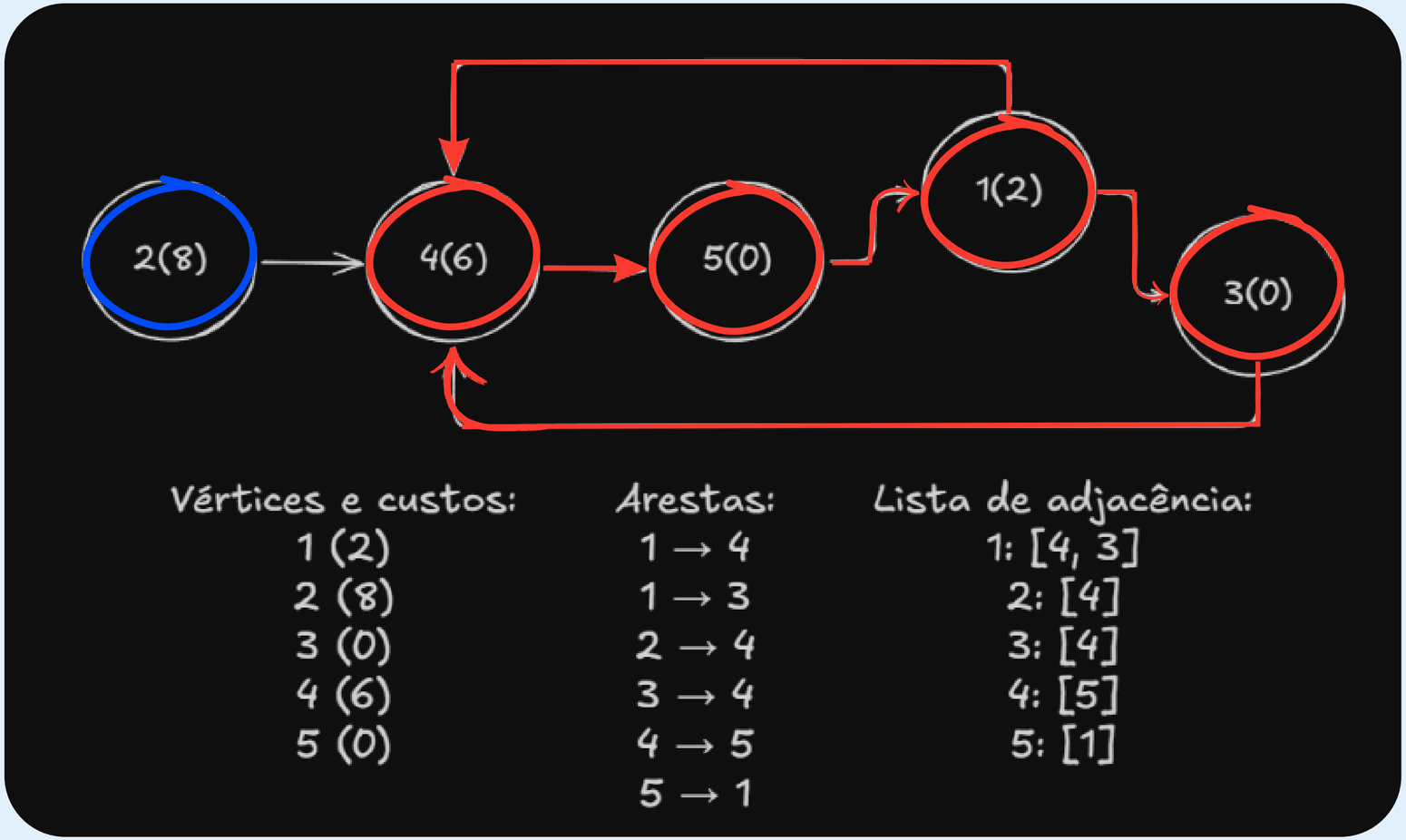
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 3, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 6: Segunda DFS no grafo original



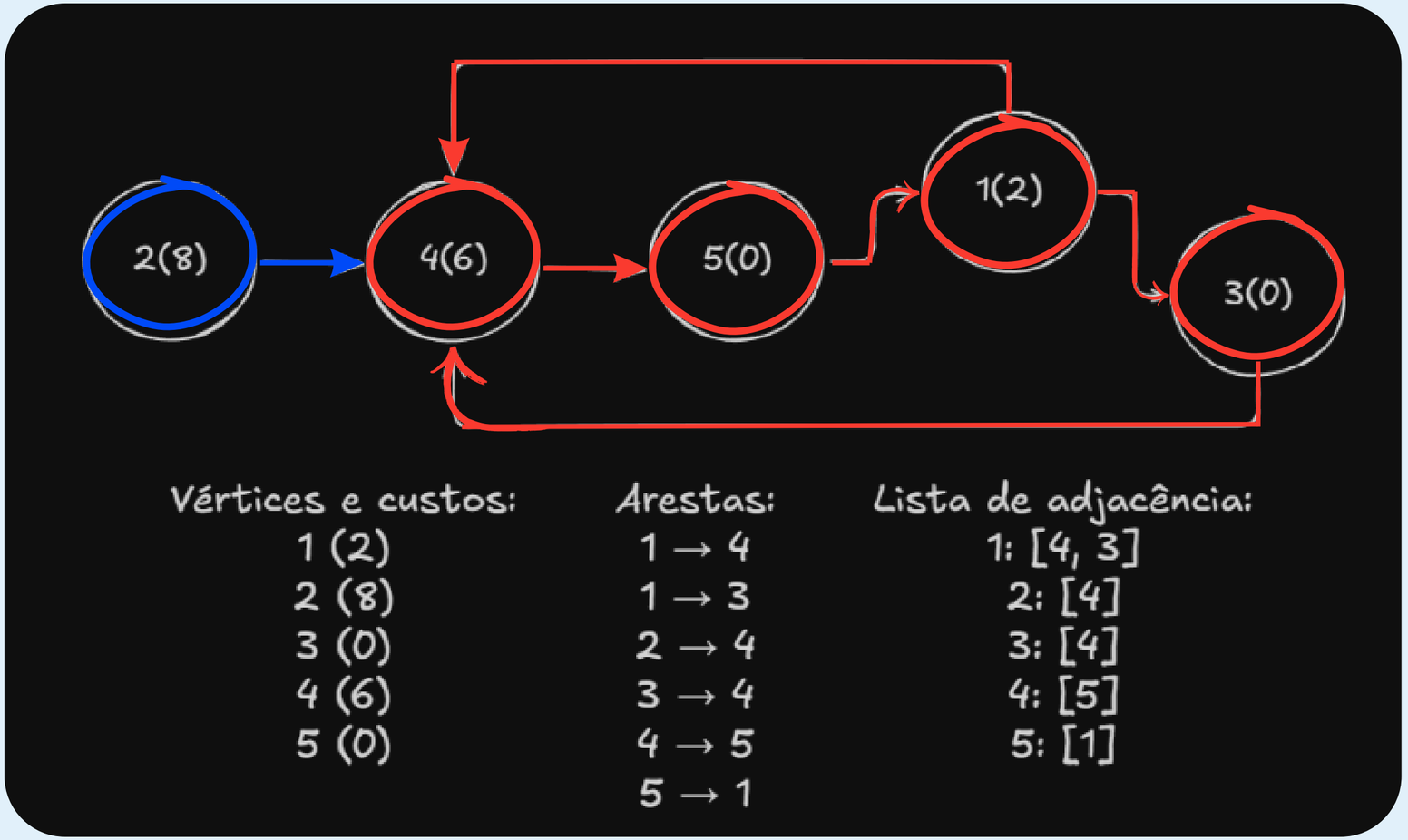
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 2, 3, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 6: Segunda DFS no grafo original



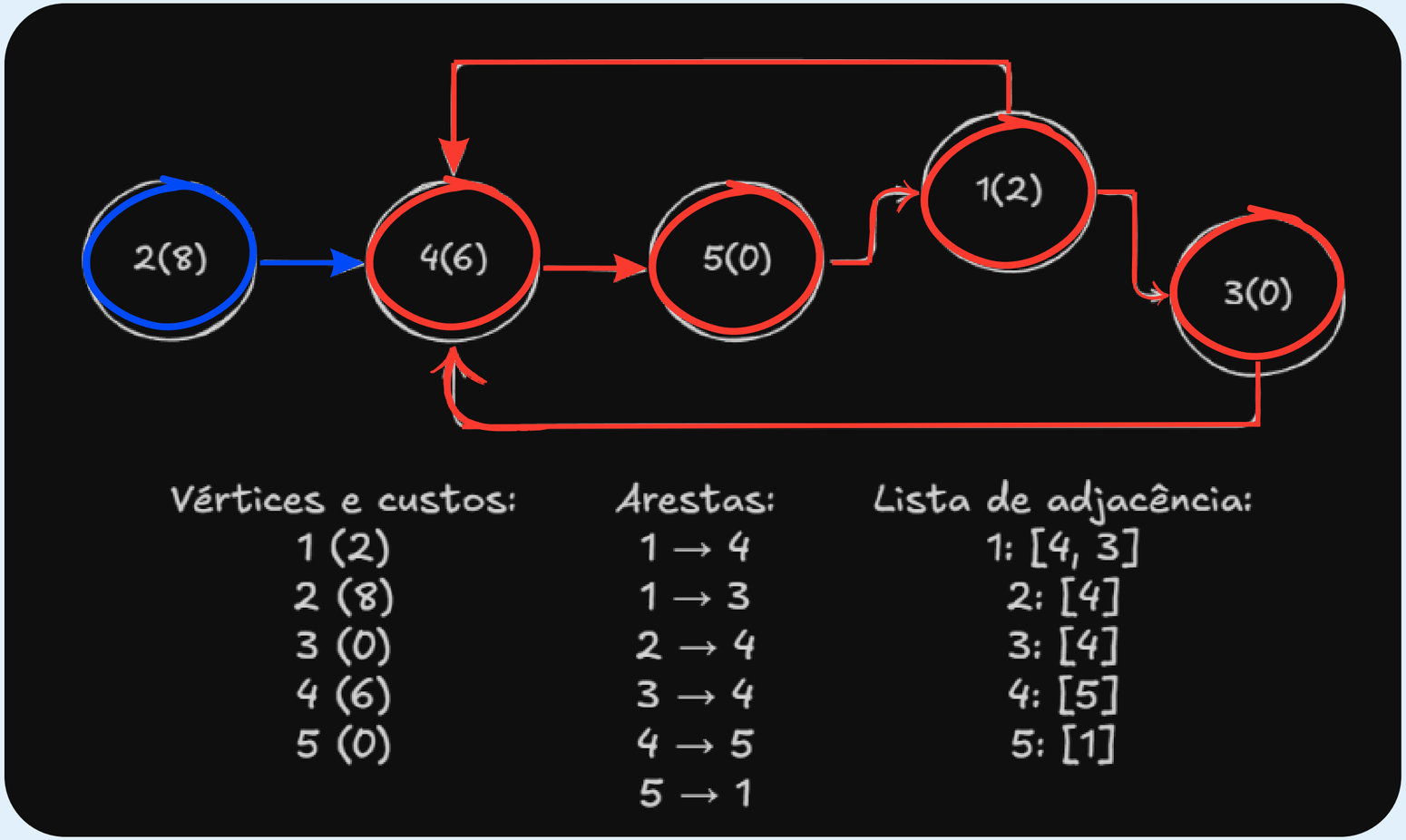
Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 2, 3, 4, 5]

Algoritmo de Kosaraju

Checkposts

Passo 6: Segunda DFS no grafo original



Pós-ordem reversa:
[1, 5, 4, 3, 2]

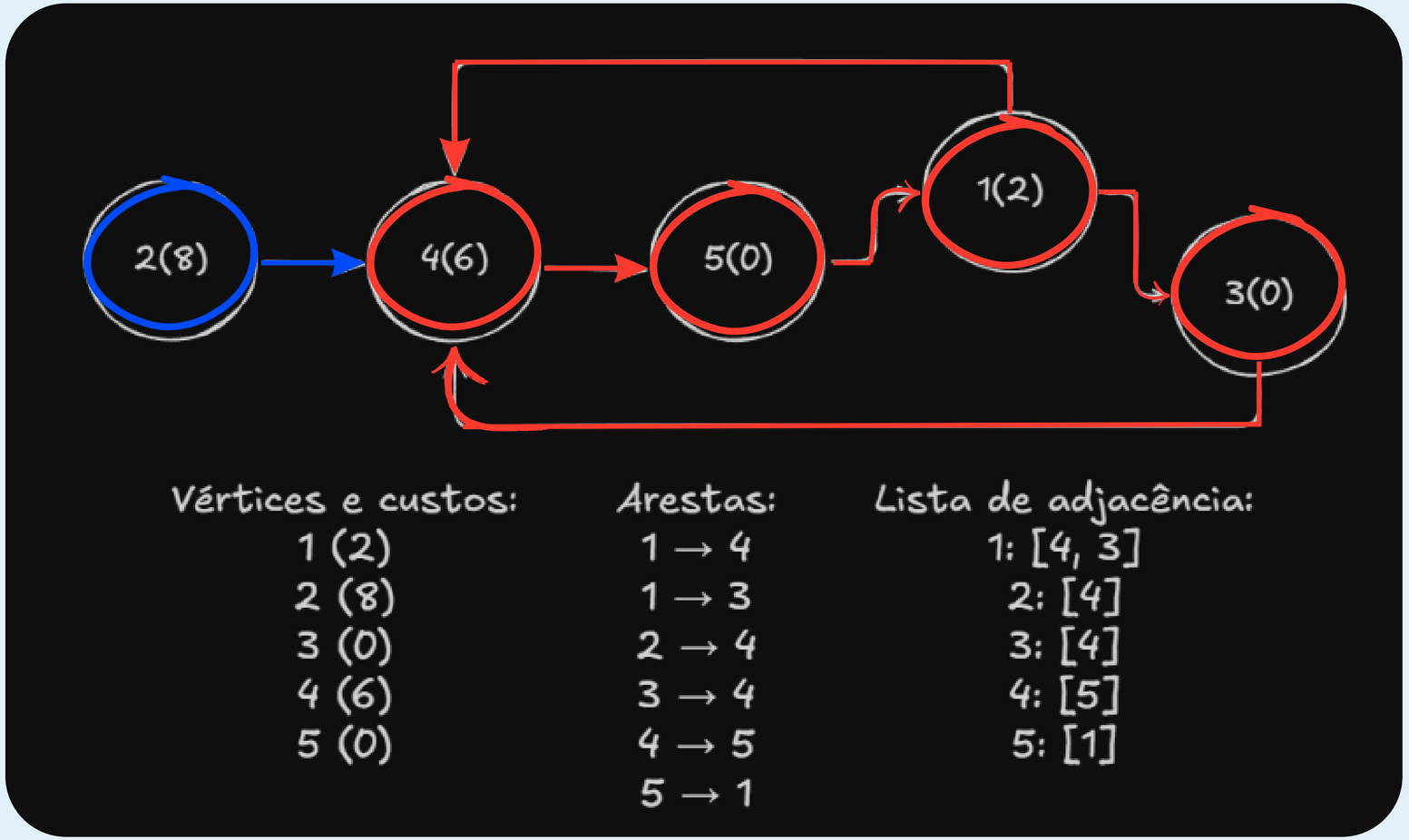
Marked:
[1, 2, 3, 4, 5]

Id = 1

Algoritmo de Kosaraju

Checkposts

Passo 6: Segunda DFS no grafo original



Pós-ordem reversa:
[1, 5, 4, 3, 2]

Marked:
[1, 2, 3, 4, 5]

Id = 1

SCC 1 = {2}

Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

SCC 0 = {1, 3, 4, 5}

SCC 1 = {2}

Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

SCC 0 = {1, 3, 4, 5}

SCC 1 = {2}

Custos:
1 → 2
2 → 8
3 → 0
4 → 6
5 → 0

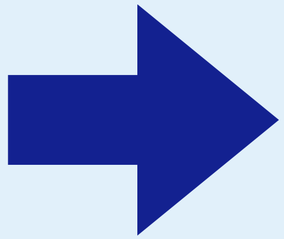
Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

SCC 0 = {1, 3, 4, 5}

SCC 1 = {2}



Custos da SCC 0 = {2, 0, 6, 0}

Custos da SCC 1 = {8}

Custos:
1 → 2
2 → 8
3 → 0
4 → 6
5 → 0

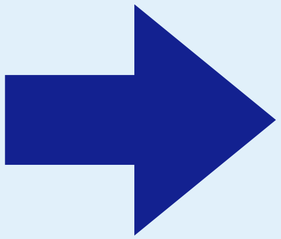
Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

SCC 0 = {1, 3, 4, 5}

SCC 1 = {2}



Custos da SCC 0 = {2, 0, 6, 0}

Custos da SCC 1 = {8}

Custos:
1 → 2
2 → 8
3 → 0
4 → 6
5 → 0

- Somar mínimo de cada SCC;
- Multiplicar quantas vezes o mínimo aparece.

Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

Custos da SCC 0 = {2, 0, 6, 0}

Custos da SCC 1 = {8}

Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

Custos da SCC 0 = {2, 0, 6, 0}

Custos da SCC 1 = {8}

- Somar mínimo de cada SCC:
- Multiplicar quantas vezes o mínimo aparece:

Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

Custos da SCC 0 = {2, 0, 6, 0}

Custos da SCC 1 = {8}

- Somar mínimo de cada SCC:
- Multiplicar quantas vezes o mínimo aparece:

1. mínimo SCC 0 = 0, aparece 2 vezes
2. mínimo SCC 1 = 8, aparece 1 vez

Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

Custos da SCC 0 = {2, 0, 6, 0}

Custos da SCC 1 = {8}

- Somar mínimo de cada SCC: Custo mínimo total: 0 + 8 = 8
- Multiplicar quantas vezes o mínimo aparece: Número de formas: 2 × 1 = 2

1. mínimo SCC 0 = 0, aparece 2 vezes
2. mínimo SCC 1 = 8, aparece 1 vez

Algoritmo de Kosaraju

Checkposts

Passo 7: aplicação das SCCs ao problema Checkposts.

Custos da SCC 0 = {2, 0, 6, 0}

Custos da SCC 1 = {8}

- Somar mínimo de cada SCC: Custo mínimo total: 0 + 8 = 8
- Multiplicar quantas vezes o mínimo aparece: Número de formas: 2 × 1 = 2

1. mínimo SCC 0 = 0, aparece 2 vezes
2. mínimo SCC 1 = 8, aparece 1 vez

input

```
5
2 8 0 6 0
6
1 4
1 3
2 4
3 4
4 5
5 1
```

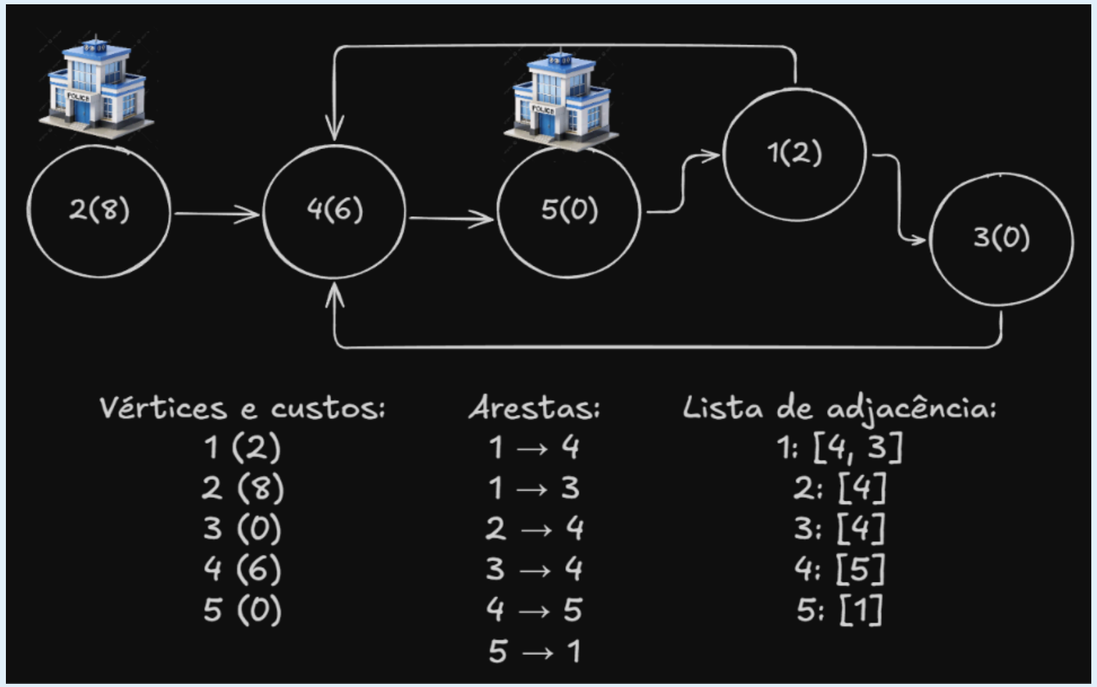
output

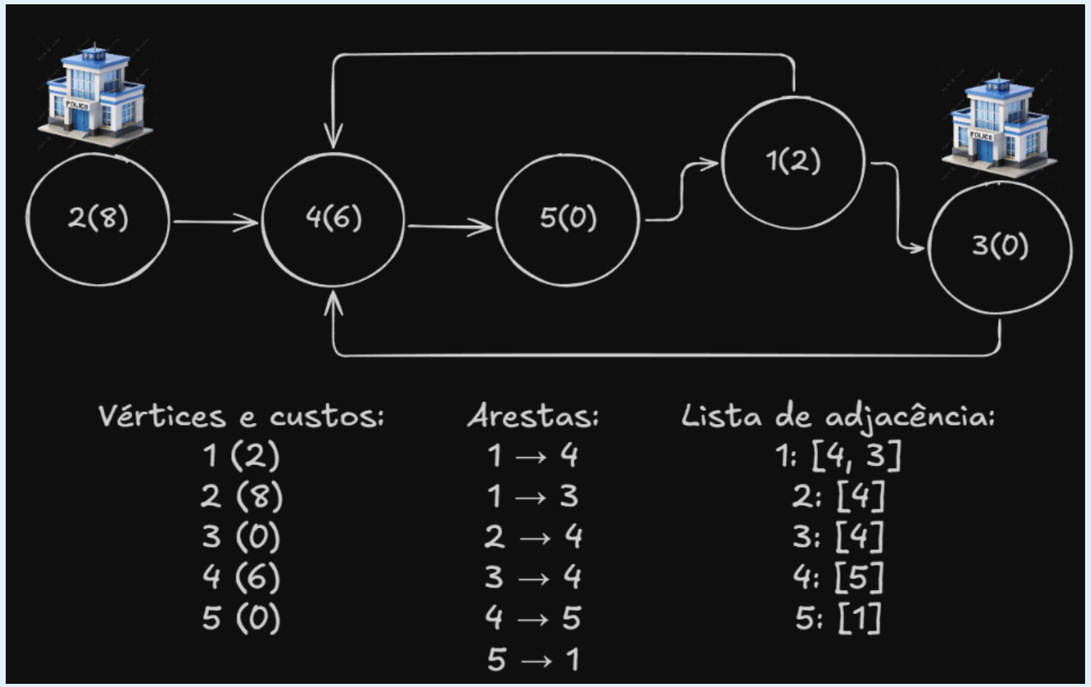
```
8 2
```

Algoritmo de Kosaraju

Checkposts

Visualização final do grafo:





Complexidades

Checkposts

Complexidade Tempo

DepthFirstOrder: *O(V+E)*

DfsOriginal: *O(V+E)*

MínimosECustos: *O(V+E)*

Complexidade Final: *O(V+E)*

Complexidade Espaço

DigrafoOriginalReverso: *O(V+E)*

DepthFirstOrder: *O(V)*

Kosaraju: *O(V)*

MínimosECustos: *O(V+E)*

Complexidade Final: *O(V+E)*


Unifor